FEATURE SPEC · UA → GA4

CRM Sync — Feature Specification Addendum

Feature specification addendum for the Universal Analytics → GA4 migration: page-view conversions become events, and consent becomes a **server-side** signal.

UA → GA4 Migration, Connected Data Streams & Trust Network DevOps

Document ID: CRM-FEAT-002 **Version:** 1.0 **Date:** 2026-05-17 **Status:** Draft — Architecture Review **Parent:** CRM-FUNC-SPEC-001 v1.2

---

1. Executive Summary

This addendum covers three interconnected initiatives:

1. **Migration** from manual CSV / Universal Analytics (UA) shaped data to real-time GA4 + multi-CDP connected streams
2. **Security & compliance hardening** with per-stream audit logging, UCP consent provenance, and agentic payment criteria
3. **Trust Network DevOps** — a non-destructive, forward-deploy model that links cross-channel configuration management to security posture, enabling safe continuous delivery across all paired data streams

The core thesis: replacing fragile, manual, CSV-driven data handoffs with authenticated, logged, consent-aware API streams — and managing the configuration of those streams through a trust-chain deployment model that makes every change auditable, reversible, and compliance-ready before it reaches production.

---

2. UA → GA4 Data Shape Migration

2.1 What Changes

DIMENSION	UA (LEGACY)	GA4 (TARGET)	IMPACT
Data model	Hit-level (pageview, event, transaction)	Event-only (all interactions are events)	Every UA custom dimension becomes a GA4 event parameter or user property
Consent	`consent_mode` v1 (basic/advanced)	`consent_mode` v2 (granular: `ad_storage`, `analytics_storage`, `ad_user_data`, `ad_personalization`)	Client-side consent banner must emit v2 signals
User identity	Client ID (cookie) + User ID (optional)	Client ID + User ID + Google Signals	Server-side events use `crm-sync.{userId}` as synthetic client_id
Custom dimensions	UA custom dimensions (index-based)	GA4 user properties (key-value, max 25)	Rename + remap all CRM properties
Measurement	`analytics.js` / `gtag.js` (client) + Measurement Protocol v1	`gtag.js` (client) + Measurement Protocol v2 (server)	Server-side MP v2 already implemented; client needs consent_mode v2
Session	Cookie-based, 30-min timeout	Event-based, configurable	Server events don't create sessions — they enrich existing ones
E-commerce	Enhanced Ecommerce (UA)	GA4 e-commerce events (`purchase`, `add_to_cart`, etc.)	Upsell events already use GA4 shape

2.2 Client-Side Consent Migration

Current state: CRM Sync's cookie consent banner writes consent to `consent_records` and fires `consent_mode` update. The signal shape needs upgrading to v2.

Target state:

```
┌──────────────────────────────────────────────────────────┐
│  Consent Banner (Webflow Embed)                          │
│                                                          │
│  User toggles → POST /auth/consent-sync                  │
│       ↓                                                  │
│  gtag('consent', 'update', {                             │
│    ad_storage: 'granted' | 'denied',                     │
│    analytics_storage: 'granted' | 'denied',              │
│    ad_user_data: 'granted' | 'denied',        ← NEW v2   │
│    ad_personalization: 'granted' | 'denied',  ← NEW v2   │
│    functionality_storage: 'granted',                     │
│    security_storage: 'granted'                           │
│  });                                                     │
│       ↓                                                  │
│  DataLayer push → GTM picks up → GA4 respects signals    │
│       ↓                                                  │
│  Worker logs to consent_records with v2 fields           │
└──────────────────────────────────────────────────────────┘
```

Mapping from CRM consent types to GA4 consent_mode v2:

CRM CONSENT TYPE	GA4 CONSENT_MODE V2 SIGNAL	DEFAULT
`consent_cookie`	`analytics_storage`	`denied`
`consent_marketing`	`ad_storage` , `ad_user_data` , `ad_personalization`	`denied`
`consent_tos`	(not a GA4 signal — logged only)	required
`consent_privacy`	(not a GA4 signal — logged only)	required
`consent_newsletter`	(not a GA4 signal — maps to user property)	optional
`consent_ccpa`	`ad_storage: denied` when opted out	optional

2.3 Server-Side User Data (Shopify → GA4)

Current state: Server-side MP v2 pushes user properties on tag mutations. Already GA4-native.

Enhancement needed: Add consent_mode v2 signals to server-side events:

```
// Current (already implemented)
user_properties: {
  crm_status: { value: "active" },
  crm_tier: { value: "vip" },
  consent_marketing: { value: "granted" }
}

// Enhanced: add consent object to event params
consent: {
  ad_storage: "denied",
  analytics_storage: "granted",
  ad_user_data: "denied",
  ad_personalization: "denied"
}
```

2.4 Migration Steps

#	STEP	OWNER	ESTIMATE
M-01	Update consent banner embed to emit consent_mode v2 signals	Worker embed	2h
M-02	Add `ad_user_data`, `ad_personalization` to `consent_records` schema	Xano	1h
M-03	Map consent_records to consent_mode v2 in `handleConsentSync`	Worker	2h
M-04	Add consent signals to server-side GA4 MP events	Worker	1h
M-05	Archive UA custom dimension mapping doc (sunset reference)	Docs	1h
M-06	Update GTM container: remove UA tags, verify GA4 tag consent settings	GTM	2h
M-07	Backfill consent_mode v2 defaults for existing users	Migration script	1h

3. Connected Data Streams — From CSV to Real-Time

3.1 The Problem with CSV

Manual CSV workflows have these failure modes:

FAILURE MODE	IMPACT	CONNECTED SOLUTION
Stale data	CSV exported → edited → re-imported hours/days later	Real-time webhook + cron sync
No audit trail	Who uploaded what, when, and what changed?	Append-only `sync_log` per stream
No consent enforcement	CSV import bypasses consent checks	Every stream checks consent state before push
Schema drift	CSV columns don't match target schema	Schema validation at ingestion + push
No rollback	Bad CSV import corrupts data	Non-destructive writes + sync_log enables replay
No entitlement check	CSV doesn't respect tier/plan	Stream-level feature gates per tenant

3.2 Per-Stream Sync Logging

Each downstream CDP gets its own sync log table, modeled after `adobe_sync_log`:

STREAM	SYNC LOG TABLE	FIELDS
Adobe AEP	`adobe_sync_log` (exists)	user_id, email_hash, ecid, dataset_id, sync_status, error_message, created_at
Salesforce	`salesforce_sync_log`	user_id, sf_contact_id, object_type, sync_status, error_message, created_at
Klaviyo	`klaviyo_sync_log`	user_id, klaviyo_profile_id, list_id, sync_status, error_message, created_at
HubSpot	`hubspot_sync_log`	user_id, hs_contact_id, sync_status, error_message, created_at
Braze	`braze_sync_log`	user_id, braze_external_id, sync_status, error_message, created_at
Attentive	`attentive_sync_log`	user_id, attentive_subscriber_id, sync_status, error_message, created_at

Every log entry records: what data was sent, to which system, whether it succeeded, and what error occurred. The UCP Dashboard can display sync history per user across all streams.

3.3 Consent-Gated Stream Architecture

```
User Mutation (tag change, form, consent toggle)
  │
  ├─► consent_records (audit — always logged)
  │
  ├─► Check consent state per stream:
  │     ├─ GA4: requires analytics_storage = granted
  │     ├─ Adobe AEP: requires adobe_aep_enabled + marketing consent
  │     ├─ Salesforce: requires salesforce_enabled + marketing consent
  │     ├─ Klaviyo: requires klaviyo_enabled + email consent
  │     ├─ HubSpot: requires hubspot_enabled + marketing consent
  │     ├─ Braze: requires braze_enabled + push/email consent
  │     └─ Attentive: requires attentive_enabled + sms consent
  │
  ├─► Push to consented streams only
  │
  └─► Log result to per-stream sync_log
```

3.4 Agentic Payment Criteria

When an AI agent makes payment or entitlement decisions, it needs auditable state:

CRITERION	SOURCE	VERIFICATION
Active subscription	`tenant:{shop}:config.plan`	Shopify Billing API
Consent state	`user_claims.*`	Real-time from Xano
Identity verified	`storefront_users.provider`	OAuth provider confirmation
Sync status	`*_sync_log`	Last successful sync per stream
Entitlement tier	`tenant:{shop}:config.tier`	Shared / Private / Enterprise
Payment method	Shopify subscription	Shopify Billing API

The agent MUST NOT process a payment or data action unless:

1. The user has an active, verified identity
2. The relevant consent is `granted` (not `denied` or missing)
3. The tenant has an active subscription at the required tier
4. The target stream sync is healthy (last sync_status = `success`)

3.5 Data Sunset Plan

#	LEGACY ITEM	SUNSET ACTION	TIMELINE
DS-01	Manual CSV customer imports	Replace with POST /sync/customers (bearer-authed)	Immediate
DS-02	UA custom dimension exports	Archive mapping doc, remove UA references from embeds	30 days
DS-03	UA-shaped consent signals (consent_mode v1)	Upgrade to v2, backfill existing users	30 days
DS-04	Direct Xano table edits (manual)	All mutations through worker API endpoints	60 days
DS-05	Non-logged data pushes	Every outbound push logged to sync_log	Immediate
DS-06	Unversioned config changes	Config changes logged with before/after diff	60 days

3.6 Logging History for UCP Compliance

The UCP Dashboard must show users:

1. **Consent history** — every consent change with timestamp, source, and which systems were notified
2. **Data flow log** — which systems have their data, when it was last synced, and the sync status
3. **Export log** — when their data was exported (data portability requests)
4. **Deletion log** — confirmation that their data was redacted from all systems

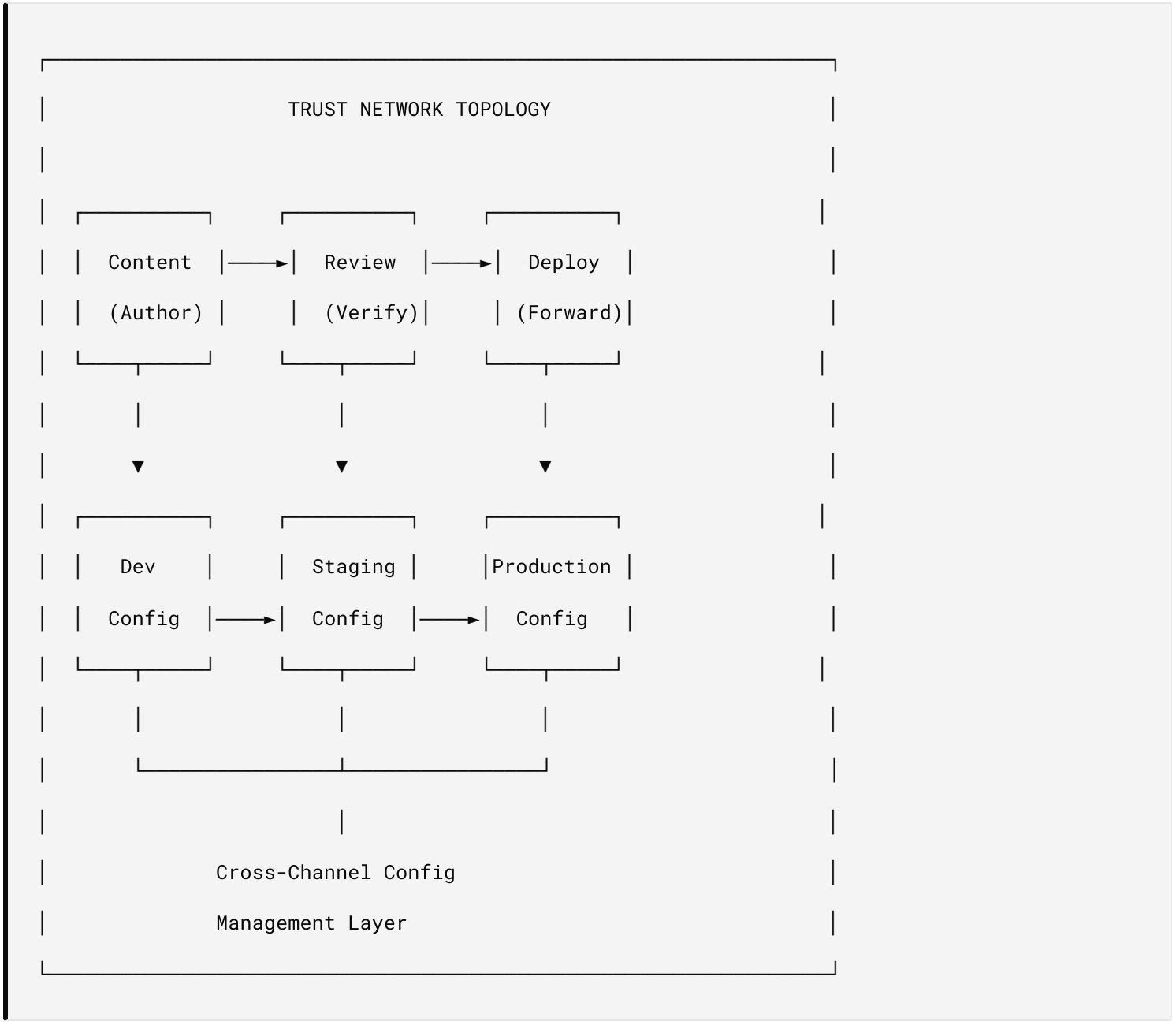
This is the basis for GDPR Art. 15 (right of access) and Art. 30 (records of processing).

---

4. Trust Network DevOps — Scaling Deploy with Linked Security

4.1 The Trust Network Model

Traditional deployment treats security as a gate at the end of the pipeline — build, test, deploy, then audit. The Trust Network inverts this: **security is the deployment topology itself**. Every node in the system (Worker, KV, Xano, Shopify, Webflow, GA4, Adobe AEP) is a trust boundary, and the deployment model ensures that changes propagate through the trust chain in a verifiable, non-destructive sequence.



4.2 Advantages of Scaling DevOps Deploy

A. CONFIGURATION-AS-CODE ACROSS ALL CHANNELS

Every integration (Shopify, Webflow, GA4, Adobe, Salesforce, etc.) is configured via the same tenant config object in KV. This means:

ADVANTAGE	HOW
Single source of config truth	`tenant:{shop}:config` holds all credentials, toggles, and stream settings
Auditable config changes	Every `POST /config` is bearer-authed and can be logged with before/after diff
Environment promotion	Dev config → staging config → production config via KV key copy, not code changes
Rollback without redeploy	Restore previous KV config value; worker code doesn't change
Multi-tenant isolation	Each shop's config is independent; one shop's misconfiguration doesn't affect others

B. LINKED SECURITY BENEFIT

When configuration management is linked to the security model, you get compounding benefits:

1. **Auth gate = deploy gate.** The same `ADMIN_KEY` that protects admin endpoints also gates config writes. If you can't authenticate, you can't deploy config. This means the deploy credential IS the security credential — there's no separate "deploy key" that can be compromised independently.
2. **Tenant isolation = blast radius containment.** A bad config deploy to `tenant:shop-a:config` cannot affect `tenant:shop-b:config`. The multi-tenant KV structure means every "deploy" (config change) is scoped to exactly one tenant. There is no global config that can break all tenants simultaneously (except `platform:config`, which is separately protected).
3. **Consent state follows config.** When you enable a new stream (e.g., `salesforce_enabled: true`), the worker immediately starts checking consent before pushing data. The security posture (consent enforcement) is embedded in the runtime, not in the deploy pipeline. You cannot accidentally deploy a stream that bypasses consent.
4. **Credential rotation is a config write.** Rotating a Shopify token, Webflow token, or Adobe credential is a `POST /config` — the same authenticated, logged,

reversible operation as any other config change. No redeploy, no downtime, no code change.

5. **Zero Trust at the edge.** Cloudflare Access (OTP) protects browser access to admin UIs. Bearer tokens protect API access. JWT protects user sessions. HMAC protects webhooks. These four layers are independent — compromising one doesn't compromise the others. And because the worker runs at the Cloudflare edge, there's no origin server to attack.

C. CROSS-CHANNEL CONFIGURATION MANAGEMENT

The key insight: every downstream system (Shopify, Webflow, GA4, Adobe, Salesforce, Klaviyo, HubSpot, Braze, Attentive) has its own credentials, its own rate limits, its own schema, and its own failure modes. Managing these independently is a combinatorial explosion. Managing them through a single config object with per-stream toggles reduces the problem to:

```
For each stream S in tenant config:
  if S.enabled AND user.consent[S.required_consent] == granted:
    push(data, S.credentials)
    log(sync_log[S], result)
```

This pattern scales linearly with the number of streams, not exponentially. Adding a new CDP (e.g., Attentive) requires:

1. Add fields to `CrmSiteConfig` interface (`attentive_enabled`, `attentive_api_key`, `attentive_subscriber_list_id`)
2. Add a `pushAttentiveEvent()` function
3. Add `attentive_sync_log` table
4. Wire into the consent-gated push chain
5. Add to the Webflow extension UI

No new auth model, no new deploy pipeline, no new security review — it inherits the existing trust network.

4.3 Non-Destructive Agile Methods

FORWARD DEPLOY (NEVER ROLLBACK CODE)

The worker is a single TypeScript file deployed to Cloudflare Workers. The deployment model is:

PRINCIPLE	IMPLEMENTATION
Forward-only deploys	Every `wrangler deploy` creates a new version. Previous versions are retained by Cloudflare. Rollback = deploy the previous version forward, not undo.
Config rollback without code rollback	Bad config? Write the old config back to KV. The worker code stays the same. Most "rollbacks" are config changes, not code changes.
Feature flags via config	`adobe_aep_enabled`, `salesforce_enabled`, etc. New features deploy in code but remain dormant until the config toggle is enabled per-tenant.
Gradual rollout	Enable a feature for one tenant first (`shop-a`), verify, then enable for all tenants. The code is already deployed; only config changes.
Non-destructive data writes	Consent records are append-only. Sync logs are append-only. Customer data is upserted (create-or-update), never delete-and-recreate.
Idempotent operations	Every sync, webhook handler, and consent write is idempotent. Running the same operation twice produces the same result. Safe to retry on failure.

THE FOUR-PHASE TRUST CYCLE

```
1. CONTENT (Author)
   └─ Developer writes code or config change
   └─ Change is scoped: which tenant, which stream, which fields
   └─ PR or config POST with description

2. DEVELOPMENT (Build + Test)
   └─ wrangler dev — local worker with real KV bindings
   └─ Webflow extension serve — local UI testing
   └─ Smoke tests (tests/smoke-test.sh) validate all auth gates
   └─ Compliance harness (tests/compliance-harness.ts) validates data contracts

3. REVIEW (Verify)
   └─ Security: all admin routes return 401 without bearer token
   └─ Privacy: no PII in logs, embeds, or client-side code
   └─ Consent: every stream push checks consent state
   └─ Config: secrets masked in GET /config
   └─ Tenant isolation: config writes scoped to tenant:{shop}

4. FORWARD DEPLOY (Ship)
   └─ wrangler deploy — immutable version created
   └─ Config write (if needed) — POST /config?shop=target
   └─ Feature toggle — enable new stream per-tenant
   └─ Monitor — /health, sync_logs, error rates
   └─ No rollback needed — fix forward with next deploy + config write
```

WHY NON-DESTRUCTIVE MATTERS

DESTRUCTIVE PATTERN	NON-DESTRUCTIVE ALTERNATIVE	BENEFIT
`DELETE FROM users WHERE ...`	PII anonymization (email → `redacted_{id}@redacted.local`)	Audit trail preserved
Drop and recreate collection	Upsert with field-level diffing	No data loss, no downtime
Force-push config	Merge new fields into existing config	Preserves credentials that aren't changing
Revert Git commit	Forward-fix in new commit	History is linear, bisectable
Delete webhook, re-register	Update webhook URL in-place	No missed events during transition
Replace all tags	Tag diff (add new, remove old)	Preserves tags from other sources

Every operation in CRM Sync is designed to be additive. The system appends consent records, upserts customer profiles, merges config fields, and logs every outbound push. The only truly destructive operation is GDPR redaction — and even that preserves the consent audit trail (as legally required).

4.4 Security Posture Scaling

As the number of connected streams grows, the security surface area grows with it. The trust network model ensures this growth is manageable:

```
Streams: 1 → 3 → 7 → 12
         │    │    │    │
         │    │    │    └─ Same bearer token auth
         │    │    └────── Same consent enforcement
         │    └─────────── Same sync_log pattern
         └──────────────── Same tenant isolation

Security work scales O(1) per new stream, not O(n).
```

Each new CDP integration inherits:

- Bearer token auth on admin endpoints (already implemented)
- Consent check before push (pattern exists)
- Sync log table (schema templated)
- Tenant-scoped credentials (config structure exists)
- Feature flag toggle (config boolean)
- PII hashing (SHA-256 helper exists)
- UCP visibility (Dashboard can read sync_logs)

The only stream-specific work is: API client code, field mapping, and error handling — the security, consent, logging, and deployment infrastructure is reused.

---

5. Implementation Roadmap

Phase 1: GA4 Consent v2 + Logging (Week 1-2)

TASK	DESCRIPTION
Upgrade consent banner to emit consent_mode v2	Add `ad_user_data`, `ad_personalization` signals
Add v2 fields to consent_records schema	Xano schema update
Server-side MP events include consent object	Worker update
Config change logging (before/after diff)	KV audit on POST /config
Rate limit auth endpoints	/auth/login, /auth/signup, /auth/forgot-password, /auth/consent-sync

Phase 2: Connected Stream Infrastructure (Week 3-4)

TASK	DESCRIPTION
Generic sync_log table creator	Admin endpoint: POST /admin/init-stream-log?stream=salesforce
Consent-gated push abstraction	`pushToStream(stream, user, data)` with consent check + logging
UCP sync history display	Dashboard shows per-user, per-stream sync status
Stream health dashboard	Admin view: last sync time, error rate, per tenant

Phase 3: CDP Integrations — Enterprise Tier (Week 5-8)

STREAM	API	AUTH	DATA SHAPE
Salesforce	REST API v59	OAuth 2.0 (JWT bearer)	Contact + Lead objects
Klaviyo	Profiles API v2024-10	API key (private)	Profile + List membership
HubSpot	Contacts API v3	OAuth 2.0 or private app	Contact properties
Braze	User Track API	REST API key	User attributes + events
Attentive	Subscribers API	API key	Subscriber + custom attributes

Phase 4: Agentic Payment + Data Sunset (Week 9-10)

TASK	DESCRIPTION
Agentic payment verification	Consent + entitlement check before payment processing
CSV import deprecation	Remove/disable manual CSV paths, redirect to API
UA reference sunset	Archive UA docs, remove UA-shaped exports
Compliance certification	UCP shows complete data flow history per user

6. React Compiler in Webflow Code Managers — Resolved Security Issues

6.1 Context

Webflow Designer Extensions (Code Managers) are React applications running inside the Webflow Designer. CRM Auth and PIM Sync both render config UIs, credential inputs, consent toggles, and sync controls. These extensions handle

sensitive state: API keys, OAuth tokens, tenant config, and consent preferences. Bringing React Compiler (automatic memoization / React Forget) into these Code Managers — compiled and served via Cloudflare Workers with Xano as the data layer — resolves an entire class of security issues that exist in hand-optimized React code.

6.2 Security Issues Resolved

A. STALE CLOSURE CREDENTIAL LEAKS

Problem: Manual `useCallback` / `useMemo` with incorrect dependency arrays create stale closures. A config form that caches an old `adminKey` value in a stale closure can send the wrong credential, or worse, send a revoked credential that was supposed to be rotated.

React Compiler fix: Automatic memoization tracks all reactive dependencies at compile time. The compiler guarantees that every closure captures current values — no stale credentials, no phantom token references.

BEFORE (MANUAL)	AFTER (REACT COMPILER)
Stale `adminKey` in `useCallback` fires request with old token	Compiler auto-tracks `adminKey` dependency — always current
Developer forgets to add `webflowToken` to dep array → sends expired token	Compiler statically analyzes all referenced variables
Config rotation requires manual audit of every `useMemo`	Zero manual dep arrays to audit

B. RE-RENDER STATE EXPOSURE

Problem: Unnecessary re-renders in config panels can briefly expose intermediate state — a half-typed API key in a text input triggers a render cycle that passes the partial value to a child component, which may log it, send it in analytics, or display it in a non-masked field.

React Compiler fix: Optimal memoization means components only re-render when their actual inputs change. Intermediate state stays contained in the component that owns it. No cascading renders that leak partial credentials down the tree.

C. SIDE-EFFECT TIMING IN OAUTH FLOWS

Problem: OAuth callback handling in Designer Extensions involves receiving tokens, storing them, and updating UI state. In hand-optimized React, `useEffect` timing bugs can cause:

- Token written to Xano *after* UI shows "connected" (race condition)
- Double-fetch of token exchange endpoint (no cleanup in strict mode)
- State update on unmounted component during redirect (memory leak + error)

React Compiler fix: The compiler understands the reactive graph and produces correctly-timed effects. Combined with Cloudflare Workers handling the actual OAuth exchange server-side, the extension UI becomes a pure display layer — it reads token status from the worker, not from local effect chains.

D. IMMUTABLE CONFIG ENFORCEMENT

Problem: Mutable state objects in React can be accidentally shared across components. If two tabs in the CRM Auth extension (Config tab and Status tab) reference the same config object and one mutates it, the other sees corrupted state. In a security context, this can mean:

- Consent toggles showing wrong state (user thinks marketing is off, but it's on)
- Config diff showing no change when credentials were actually modified
- Sync status reflecting a different tenant's state

React Compiler fix: React Compiler requires (and enforces at compile time) immutable data patterns. Mutations are flagged as errors during compilation.

Every state update produces a new object, so cross-component state corruption is structurally impossible.

E. BUNDLE INTEGRITY VIA COMPILE-TIME VALIDATION

Problem: Webflow Designer Extensions ship as a `bundle.zip` containing compiled JS. Without compile-time validation, runtime bugs in production are invisible until a user hits them — and in a security-sensitive Code Manager, "runtime bug" can mean "credential sent to wrong endpoint" or "consent state not checked."

React Compiler fix: The compilation step acts as a static analysis gate. Code that violates React's rules of hooks, mutates state directly, or creates non-deterministic renders fails compilation. This means:

```
TypeScript compile → React Compiler validate → Bundle → Upload to Webflow
                              ↑
                     Security gate:
                     - No stale closures
                     - No mutable state sharing
                     - No effect timing bugs
                     - No hook rule violations
```

The bundle that reaches Webflow Designer has been structurally verified — not just type-checked, but semantically validated for correct reactive behavior.

6.3 Cloudflare Workers as Secure Computation Boundary

React Compiler in the extension UI is half the story. The other half is what code runs *in the Worker* vs. what runs *in the extension*:

CONCERN	RUNS IN EXTENSION (REACT)	RUNS IN WORKER (CLOUDFLARE)
Credential storage	Never — reads masked config from worker	KV encrypted at rest
OAuth token exchange	Never — redirect goes to worker callback	Worker validates state, exchanges code for token
Consent enforcement	Display only — shows current state	Enforced before every outbound push
Config writes	Sends form data to worker API	Worker validates, merges, writes to KV
PII handling	Never sees raw PII	SHA-256 hash before any external push
Sync execution	Triggers via button → POST to worker	Worker runs sync with tenant isolation

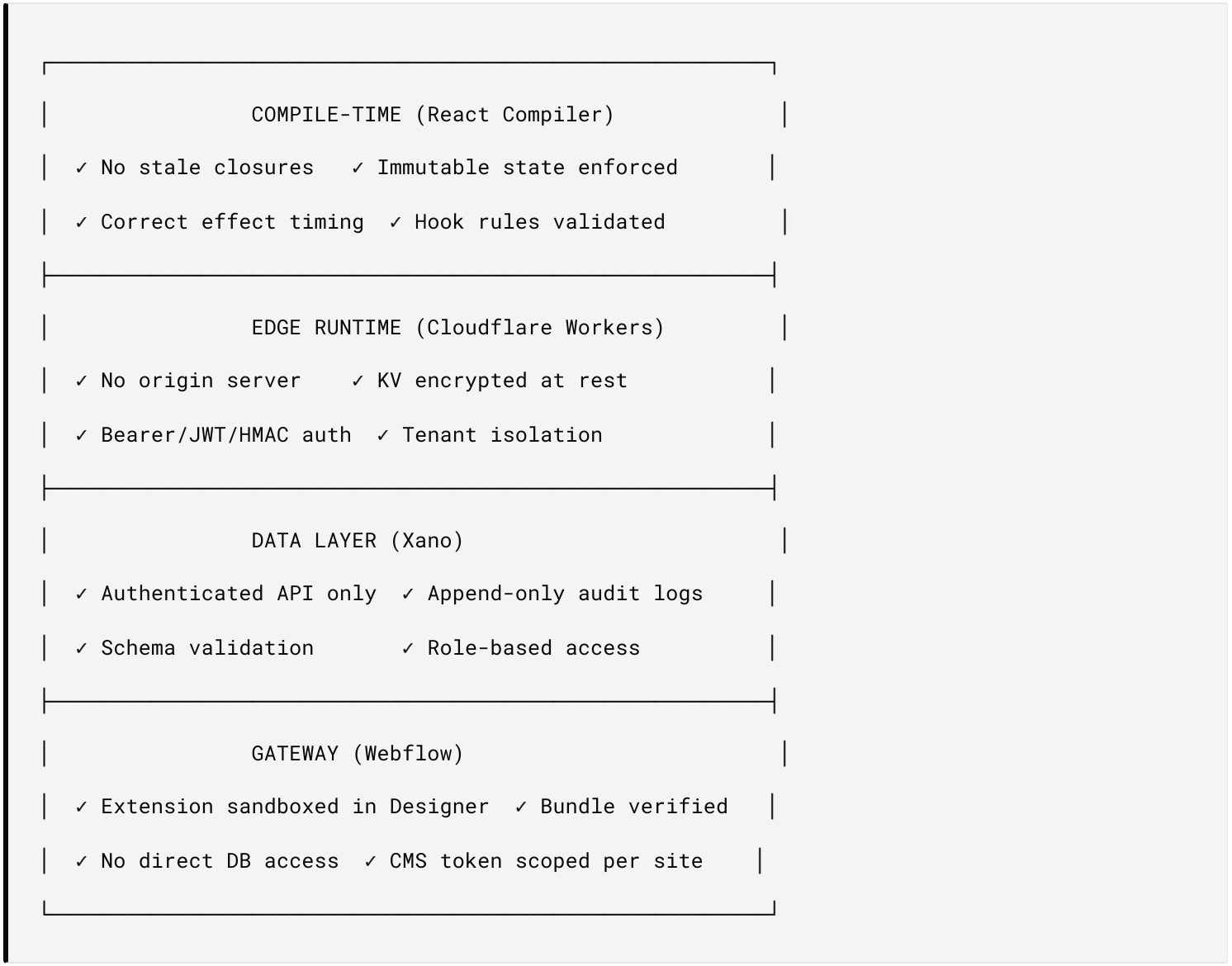
The React Compiler ensures the extension UI is a **correct, minimal display layer**. Cloudflare Workers ensure all **security-critical computation happens server-side**. Xano ensures all **data persistence is behind authenticated API calls**. No single layer can compromise the system alone.

6.4 Xano Tool Integration Security

Xano serves as the authenticated data layer behind both the Worker and the extension. React Compiler improves the extension↔Xano interaction:

PATTERN	WITHOUT COMPILER	WITH COMPILER
Xano API key in fetch call	Stale closure may cache old key	Always uses current key from props/context
Retry logic on Xano 401	Manual `useEffect` cleanup can leak retries	Compiler-managed effects cancel cleanly
Optimistic UI on Xano write	Rollback may not fire if component unmounts	Compiler ensures cleanup runs
Xano pagination state	Mutable cursor can cause duplicate fetches	Immutable cursor state prevents double-load

6.5 Summary: Compound Security Stack



Each layer resolves a different attack surface. React Compiler eliminates the **UI-layer logic bugs** that traditional React security tooling cannot catch because they're semantic, not syntactic. The Worker eliminates **server-side exposure**. Xano eliminates **direct data access**. Webflow's extension sandbox eliminates **cross-site interference**. Together, they create a defense-in-depth stack where a vulnerability in one layer cannot cascade.

---

7. Supply Chain Trust Partner Risk — Design over Configuration

7.1 The Supply Chain Risk Landscape

Recent high-profile breaches (SolarWinds, Codecov, MOVEit, xz-utils, Polyfill.io, 3CX) share a common pattern: **trust in a third-party dependency or partner became the attack vector**. Organizations with large partner ecosystems — e-commerce brands connected to CDPs, payment processors, analytics platforms, and marketing automation tools — face compounding supply chain risk. Every partner integration is a trust boundary. Every API credential stored is a potential compromise vector. Every unaudited data flow is a compliance liability.

Traditional configuration-heavy approaches amplify this risk:

RISK FACTOR	CONFIGURATION-HEAVY APPROACH	DESIGN-OVER-CONFIGURATION APPROACH
Credential sprawl	Each partner needs keys stored in env vars, config files, CI secrets — often duplicated across environments	Single tenant config object in encrypted KV; credentials never touch code, repos, or CI pipelines
Dependency supply chain	npm packages with transitive deps (avg. 300+ packages) — each a potential compromise point	Single-file Worker with zero npm runtime deps; React Compiler validates at build, not runtime
Configuration drift	Partner configs diverge across dev/staging/prod; manual sync required	Config-as-code in KV with environment promotion (copy key, not code)
Audit gap	Who changed what partner config, when? Often no log.	Every `POST /config` is bearer-authed with before/after diff logged
Blast radius	One compromised config can affect all tenants	Tenant-isolated KV keys — breach of `tenant:shop-a:config` cannot read `tenant:shop-b:config`
Partner offboarding	Revoking a partner requires finding all places their credential is stored	Single KV key per tenant; set `{stream}_enabled: false` and credential is never read again

7.2 Why Distributed Code + Design Simplicity Closes the Gap

CRM Sync's architecture is intentionally minimal at each layer:

```
┌──────────────────────────────────────────────────────────────┐
│                  SIMPLIFIED DISTRIBUTED CODE                 │
│                                                              │
│  Extension (React Compiler)                                  │
│  └─ Zero runtime deps in bundle                              │
│  └─ Compile-time validation = no runtime surprise            │
│  └─ Sandboxed in Webflow Designer = no cross-origin access   │
│                                                              │
│  Worker (Cloudflare, single file)                            │
│  └─ Zero npm runtime dependencies                            │
│  └─ No node_modules in production                            │
│  └─ No origin server = no server to patch                    │
│  └─ V8 isolate = no shared memory between requests           │
│                                                              │
│  Data (Xano)                                                 │
│  └─ No direct SQL access from any client                     │
│  └─ API-only with auth on every endpoint                     │
│  └─ Schema enforced at the platform level                    │
│                                                              │
│  Gateway (Webflow / Shopify)                                 │
│  └─ OAuth tokens scoped to minimum required permissions      │
│  └─ Webhook verification (HMAC / state nonce)                │
│  └─ Platform-managed TLS and DDoS protection                 │
└──────────────────────────────────────────────────────────────┘
```

Design over configuration means the system's security properties are structural, not configurable. You cannot misconfigure the Worker into accepting unsigned webhooks — the HMAC check is compiled into the handler. You cannot accidentally expose PII to GA4 — the SHA-256 hash is in the code path,

not a config toggle. You cannot bypass consent — the consent check is in the push function, not a flag you can turn off.

7.3 Supply Chain Trust Partner Risk Matrix

For each trust partner in the CRM Sync ecosystem, here is the specific risk and how the architecture mitigates it:

TRUST PARTNER	SUPPLY CHAIN RISK	MITIGATION
Shopify	Compromised Admin token grants customer data access	Token stored in per-tenant KV (not env vars); auto-refresh before expiry; scoped to minimum permissions; HMAC validates all inbound webhooks
Webflow	Compromised CMS token allows data injection into published site	OAuth token scoped per site; webhook state nonce prevents replay; CMS writes validate schema before push
Xano	Compromised API key gives access to all customer records	API key per tenant; role-based endpoint access; no direct SQL; append-only audit tables
GA4	API secret exfiltration allows event injection	Secret stored in KV config (not code); only category/consent data sent (no PII); synthetic client_id prevents user enumeration
Adobe AEP	IMS OAuth compromise allows CDP profile manipulation	Client credentials flow with short-lived tokens (cached in KV with TTL); all PII SHA-256 hashed before transmission
Resend	API key compromise allows sending email as the brand	Key stored as wrangler secret; only two email templates (welcome, reset); tokens are single-use with TTL
Cloudflare	KV compromise exposes all tenant configs	KV encrypted at rest (Cloudflare-managed); secrets masked in API responses; Access Zero Trust with OTP on admin URLs
npm ecosystem	Malicious package in dependency tree	Zero runtime npm deps in Worker; build-only deps for TypeScript compilation; React Compiler catches semantic issues at build time

7.4 Design Principles for Partner Risk Reduction

PRINCIPLE 1: NO TRANSITIVE TRUST

Every partner integration is **direct** — the Worker talks to Shopify's API, not through a third-party SDK that wraps Shopify's API. This eliminates transitive dependency risk. The Worker uses `fetch()` (built into the runtime) and `crypto.subtle` (Web Crypto API, built into V8). No axios, no node-fetch, no lodash, no moment.

```
Traditional:  App → SDK → HTTP lib → TLS lib → Partner API
              ↑       ↑         ↑          ↑
              4 supply chain trust points

CRM Sync:    Worker → fetch() → Partner API
              ↑
              0 third-party trust points in the runtime path
```

PRINCIPLE 2: CREDENTIAL BLAST RADIUS = 1 TENANT

A compromised credential in a traditional multi-tenant system with shared env vars affects all tenants. In CRM Sync, every credential lives in `tenant:{shop}:config`. Compromising one tenant's Shopify token gives access to one shop's customer data — not all shops. The attacker must breach KV access + know the specific tenant key + bypass bearer token auth on the config endpoint.

PRINCIPLE 3: DESIGN-TIME ENFORCEMENT > RUNTIME CONFIGURATION

SECURITY PROPERTY	CONFIGURATION APPROACH (RISKY)	DESIGN APPROACH (CRM SYNC)
PII hashing	`config.hashPii = true` (can be set to false)	`SHA256(email)` hardcoded in `pushAdobeEvent()` — cannot be disabled
Consent check	`config.enforceConsent = true` (can be toggled)	`if (!user.consent[stream.required]) return` — structural in every push function
Webhook verification	`config.verifyWebhooks = true` (can be skipped)	HMAC check is the first line of every webhook handler — no toggle exists
Token masking	`config.maskSecrets = true` (can be turned off)	`getPublicCrmConfig()` always strips secrets — no "show secrets" mode
Tenant isolation	Namespace configured per deployment	KV key prefix `tenant:{shop}:` is in the function signature — impossible to cross

PRINCIPLE 4: ON-DEMAND DEVELOPMENT REDUCES EXPOSURE WINDOW

Configuration-on-demand (traditional) means partners are always connected, always have valid credentials, always have access — even when no sync is running. Design-on-demand (CRM Sync) means:

- **Feature-flagged streams**: `salesforce_enabled: false` means the Salesforce credential is never read from config, the push function is never called, and no network request is made. The integration exists in code but is inert.
- **Short-lived tokens**: Adobe IMS tokens are cached with TTL and re-fetched on demand. Shopify tokens auto-refresh. OAuth state nonces expire in 10 minutes. There is no long-lived session to hijack.
- **Cron-driven sync**: Customer sync runs every 15 minutes via cron. Between syncs, no persistent connection exists to any partner API. The attack surface is intermittent, not continuous.

7.5 Organizational Impact

For organizations evaluating CRM Sync against traditional integration platforms:

CONCERN	TRADITIONAL IPAAS / MIDDLEWARE	CRM SYNC ARCHITECTURE
SOC 2 audit surface	Middleware vendor + all partner SDKs + CI/CD secrets + container runtime	Cloudflare (SOC 2 Type II) + Xano (API platform) + zero runtime deps
Vendor lock-in risk	Middleware vendor controls data flow; migration = rewrite	Worker is standard TypeScript + `fetch()`; any edge runtime can host it
Partner onboarding	Install SDK, store credentials in vault, configure middleware rules	Add fields to `CrmSiteConfig`, write one `push{Partner}()` function
Partner offboarding	Find all credential references, revoke across environments	Set `{partner}_enabled: false` in one KV key
Incident response time	Debug through middleware logs + SDK internals + partner dashboards	Single Worker log stream + per-stream sync_log in Xano
Compliance officer review	Multiple systems, multiple credential stores, multiple audit logs	One config endpoint, one consent table, one sync_log per stream

The architecture is designed so that a compliance officer or security auditor can answer three questions in under 5 minutes:

1. "**Where are credentials stored?**" → `tenant:{shop}:config` in Cloudflare KV, encrypted at rest, masked in API responses.
2. "**Where does customer data go?**" → Only to streams where `{stream}_enabled = true` AND user consent is `granted`. Every push is logged to `{stream}_sync_log`.
3. "**What happens if a partner is compromised?**" → Set `{stream}_enabled: false` via `POST /config`. Credential is never read again. No code change, no redeploy, < 30 seconds.

8. Tool Architecture — Client-Side, Compile-Time & Dynamic Server

CRM Sync's stack divides cleanly into three execution phases. Each phase has distinct tools, and the security/trust guarantees differ at each phase. Understanding which tool operates where — and what it can and cannot do — is essential for both development and auditing.

8.1 Phase Map

```
┌──────────────────────────────────────────────────────────────────┐
│  CLIENT SIDE (Browser / Webflow Designer)                        │
│  Runs: in user's browser or Webflow Designer sandbox             │
│  Trust: untrusted — all input must be validated server-side      │
│                                                                  │
│  Tools:                                                          │
│  ├─ Webflow Components (Designer Extensions UI)                  │
│  ├─ Webflow Semantic Design (styles, variables, classes)         │
│  ├─ Webflow Publish APIs (site publish, CMS push)                │
│  ├─ Consent banner + DataLayer (gtag consent_mode v2)            │
│  ├─ Embed scripts (UCP dashboard, footer, compliance page)       │
│  ├─ Installable PWA (/configure shell + manifest + service worker)  │
│  └─ Cross-device event bus (crm-events.js → GTM / GA4 / Merchant)   │
├──────────────────────────────────────────────────────────────────┤
│  COMPILE TIME (Build / CI)                                       │
│  Runs: developer machine or CI pipeline, before deploy           │
│  Trust: verified — output is the production artifact             │
│                                                                  │
│  Tools:                                                          │
│  ├─ TypeScript Compiler (tsc — type checking)                    │
│  ├─ React Compiler (automatic memoization, semantic validation)     │
│  ├─ Wrangler (Cloudflare Workers build + deploy)                 │
│  ├─ Webflow Extension Bundler (webflow extension bundle)         │
│  ├─ Compliance Harness (tests/compliance-harness.ts)             │
│  └─ Capacitor + Electron (native build — iOS / Android / desktop)   │
├──────────────────────────────────────────────────────────────────┤
│  DYNAMIC SERVER (Edge Runtime / API)                             │
│  Runs: Cloudflare Workers V8 isolate or Xano API runtime         │
```

```
│  Trust: trusted — authenticated, isolated, encrypted                │
│                                                                     │
│  Tools:                                                             │
│  ├─ Cloudflare Workers (request handling, routing, auth)            │
│  ├─ PWA shell + manifest + SW + /get + /crm-events.js + /edge/geo   │
│  ├─ Cloudflare KV (config storage, state, token cache)              │
│  ├─ Wrangler Secrets (ADMIN_KEY, JWT_SECRET, API keys)              │
│  ├─ Xano Auth (user registration, login, JWT issuance)              │
│  ├─ Xano API (CRUD on storefront_users, consent_records, tags)      │
│  └─ Xano Functions (server-side logic, schema validation)           │
└─────────────────────────────────────────────────────────────────────┘
```

8.2 Client-Side Tools — Features & Functions

WEBFLOW COMPONENTS (DESIGNER EXTENSIONS)

FEATURE	FUNCTION	SECURITY BOUNDARY
CRM Auth Extension	Config panel: enter worker URL, API keys, toggle streams	Keys sent to Worker via `POST /config` — never stored client-side
Tab UI (Config, Auth, Embeds, Plan, Status)	Navigate extension features without page load	React state stays in Designer sandbox — no cross-origin access
Consent toggles	Display current consent state per user	Read-only from Worker `/auth/me` — cannot mutate directly
Sync trigger buttons	Fire `POST /sync/customers` or `POST /sync/webflow`	Button sends bearer-authed request — Worker validates before executing
Status display	Show last sync time, error count, tenant health	Polls Worker `/health` and `/config` — secrets are masked in response

WEBFLOW SEMANTIC DESIGN

FEATURE	FUNCTION	CRM SYNC USAGE
Design tokens / variables	CSS custom properties managed in Webflow	Consent banner and embed pages reference site-wide tokens for consistent branding
Component classes	Reusable styled elements	UCP dashboard components (consent history table, sync status cards) use shared classes
Conditional visibility	Show/hide elements based on CMS field state	CMS-driven pages show/hide sections based on `status`, `consent-marketing`, or `tags` fields
Style inheritance	Parent → child style cascade	Embed HTML inherits site styles when injected via `<script>` tags

WEBFLOW PUBLISH APIS

FEATURE	FUNCTION	CRM SYNC USAGE
Site publish	Push staged changes to live CDN	After CMS fields are updated by Worker sync, publish propagates changes to live site
CMS Collection CRUD	Create/read/update/delete CMS items via API	Worker uses CMS API to create/update customer profiles in `storefront-users` collection
CMS Field Management	Ensure required fields exist on collection	`POST /admin/webflow-ensure-fields` creates missing fields (consent, Adobe, commerce)
Webhook registration	Subscribe to CMS item changes	`POST /admin/register-webhooks` registers `item_changed` webhook for bidirectional sync
Collection schema	Read field definitions for a collection	Worker validates field existence before attempting CMS writes

8.3 Compile-Time Tools — Features & Functions

TYPESCRIPT COMPILER (TSC)

FEATURE	FUNCTION	SECURITY VALUE
Static type checking	Verify types match at every boundary	Prevents passing a `string` where `CrmSiteConfig` is expected — catches config shape mismatches before deploy
Interface enforcement	`CrmSiteConfig`, `PlatformConfig`, `Env` interfaces	Every config access is type-checked — cannot read a field that doesn't exist
Strict null checks	`strictNullChecks: true`	Forces explicit handling of missing config fields, missing consent values, null API responses
No implicit any	`noImplicitAny: true`	Every variable has a known type — no untyped data flows through the Worker

REACT COMPILER

FEATURE	FUNCTION	SECURITY VALUE
Automatic memoization	Compiler inserts `useMemo` / `useCallback` with correct deps	Eliminates stale closure bugs (see Section 6.2A)
Immutability enforcement	Rejects mutations of state/props at compile time	Prevents cross-component state corruption in extension UI
Effect validation	Verifies effect dependencies and cleanup	Prevents OAuth token exchange race conditions
Hook rules check	Validates rules of hooks at compile time	Catches conditional hook calls that could crash the extension

WRANGLER (CLOUDFLARE CLI)

FEATURE	FUNCTION	CRM SYNC USAGE
`wrangler dev`	Local Worker development server with KV bindings	Test all routes locally before deploy
`wrangler deploy`	Build TypeScript → deploy to Cloudflare edge	Immutable versioned deploy — previous versions retained
`wrangler secret put`	Store secrets in Worker environment	`ADMIN_KEY`, `JWT_SECRET`, `SHOPIFY_APP_SECRET` — never in code
`wrangler kv:*`	KV namespace CRUD operations	Migration scripts, tenant setup, config inspection
`wrangler tail`	Live log streaming from deployed Worker	Debug production issues without accessing KV directly

WEBFLOW EXTENSION BUNDLER

FEATURE	FUNCTION	SECURITY VALUE
`webflow extension bundle`	Compiles TypeScript → JS, packages as `bundle.zip`	Produces a verified artifact for upload to Webflow
Bundle size gate	Small bundles (~12KB) indicate minimal dependencies	Large bundle = unexpected dependency = potential supply chain risk
Static asset inclusion	HTML, CSS, JS all bundled together	No runtime CDN fetches from third-party origins

8.4 Dynamic Server Tools — Features & Functions

CLOUDFLARE WORKERS

FEATURE	FUNCTION	CRM SYNC USAGE
V8 isolate execution	Each request runs in isolated V8 context — no shared memory	Tenant A's request cannot read Tenant B's in-flight data
`fetch()` **(built-in)**	HTTP client — no third-party HTTP library	All partner API calls (Shopify, Xano, GA4, Adobe, Resend) use native `fetch()`
`crypto.subtle`	Web Crypto API — SHA-256, HMAC, key derivation	PII hashing for Adobe AEP, HMAC verification on Shopify webhooks, JWT signing
`scheduled()` **handler**	Cron-triggered function — runs every 15 min	Iterates tenant registry, runs customer sync for each shop
Request routing	URL pattern matching in `fetch()` handler	62 routes mapped to auth-gated handlers
Response headers	Custom headers on every response	CORS, `Set-Cookie` (httpOnly JWT), `X-Content-Type-Options`

CLOUDFLARE KV

FEATURE	FUNCTION	CRM SYNC USAGE
Key-value storage	String → string/JSON, with optional TTL	Tenant configs, OAuth state, PKCE verifiers, reset tokens, Adobe tokens
Encryption at rest	Cloudflare-managed encryption	All credentials in KV are encrypted — no plaintext on disk
TTL expiry	Automatic deletion after time period	OAuth state (10 min), PKCE (5 min), reset tokens (1h/24h), Adobe tokens (~24h)
Namespace isolation	KV binding `CRM_STATE` scoped to this Worker	Other Workers on the same account cannot read CRM config
Global replication	KV data replicated across Cloudflare edge	Config reads are fast from any edge location — no single region bottleneck

WRANGLER SECRETS

FEATURE	FUNCTION	CRM SYNC USAGE
Environment secrets	Encrypted at rest, injected at runtime via `env.*`	`ADMIN_KEY`, `JWT_SECRET`, `SHOPIFY_APP_SECRET`, `XANO_API_KEY`
Not in code	Secrets never appear in source, wrangler.toml, or KV	Eliminates accidental credential commit to Git
Per-environment	Different secret values for preview vs. production	Dev and prod use different keys — dev compromise doesn't affect prod

XANO AUTH

FEATURE	FUNCTION	CRM SYNC USAGE
User registration	Create `storefront_users` record with hashed password	`POST /auth/signup` → Xano creates user → Worker issues JWT
Login + JWT	Verify credentials, return signed JWT	`POST /auth/login` → Xano validates → Worker sets httpOnly cookie
Password reset	Generate reset token, verify on submission	Worker generates KV-stored token → Resend emails link → Xano updates password
OAuth user creation	Create user from Google/Shopify/Webflow OAuth profile	OAuth callback → Worker creates/updates Xano user with provider info
Session validation	Verify JWT on every authenticated request	Worker decodes JWT from cookie, loads user from Xano, checks expiry

XANO API

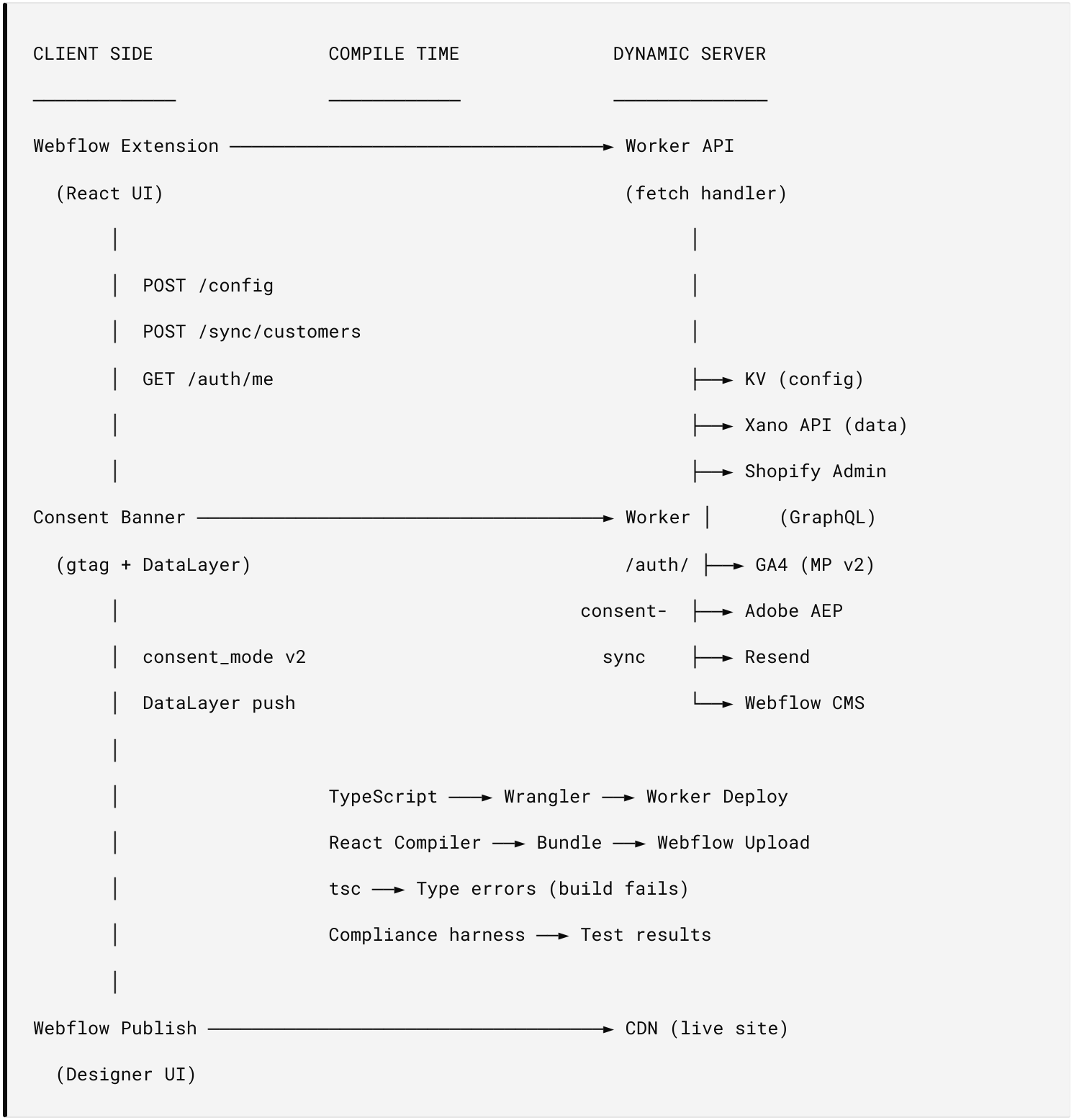
FEATURE	FUNCTION	CRM SYNC USAGE
`storefront_users` **CRUD**	Customer profile management	Shopify sync creates/updates users; Webflow sync reads users for CMS push
`consent_records` **append**	Immutable consent audit log	Every consent change logged with timestamp, source, action, user_id
`user_tag_map` **join table**	Many-to-many user↔tag relationships	Tag mutations update join table; tags propagated to Webflow CMS + Shopify metafields
`user_claims` **table**	Consent state per user (tos, privacy, cookie, marketing)	Read before every stream push to enforce consent-gated architecture
`user_extras` **table**	Adobe ECID, sync status, identity graph ID	Written after Adobe AEP sync; read by Webflow CMS sync for field population
`adobe_sync_log`	Per-user Adobe sync audit trail	Append on every AEP push attempt — success or failure with error message

XANO FUNCTIONS

FEATURE	FUNCTION	CRM SYNC USAGE
Server-side logic	Business rules that run in Xano's runtime	Consent state derivation, tag aggregation, user merge logic
Schema validation	Enforce field types and constraints	Prevents malformed data from reaching the database — Worker validates before write, Xano enforces at persist
Triggers	Post-write hooks on tables	After `storefront_users` update, trigger downstream notifications
Bulk operations	Batch CRUD across multiple records	Shopify customer sync pushes multiple users in single batch

8.5 Tool Interaction Matrix

How the three phases connect — which tool calls which, and what data crosses the boundary:


CLIENT SIDE
COMPILE TIME
DYNAMIC SERVER
Webflow Extension
(React UI)
Worker API
(fetch handler)
POST /config
POST /sync/customers
GET /auth/me
KV (config)
Xano API (data)
Shopify Admin
Consent Banner
(gtag + DataLayer)
Worker
/auth/
consent-sync
(GraphQL)
GA4 (MP v2)
Adobe AEP
Resend
Webflow CMS
consent_mode v2
DataLayer push
TypeScript → Wrangler → Worker Deploy
React Compiler → Bundle → Webflow Upload
tsc → Type errors (build fails)
Compliance harness → Test results
Webflow Publish
(Designer UI)
CDN (live site)

8.6 Feature Gates by Execution Phase

CAPABILITY	CLIENT SIDE	COMPILE TIME	DYNAMIC SERVER
Read credentials	Never	Never (type-checks only)	Yes (KV + secrets)
Write customer data	Never	Never	Yes (Xano API)
Verify webhooks	Never	Never	Yes (HMAC via `crypto.subtle`)
Issue JWT	Never	Never	Yes (Worker signs with `JWT_SECRET`)
Check consent	Display only	Never	Enforce (block push if denied)
Trigger sync	Button click → API call	Never	Execute (fetch Shopify → write Xano → push CMS)
Modify config	Form input → API call	Never	Write to KV (bearer-authed)
Hash PII	Never	Never	Yes (SHA-256 before external push)
View secrets	Masked values only	Never	Available via `env.*` at runtime
Deploy code	Never	Yes (Wrangler)	Self (V8 isolate loads deployed code)
Bundle extension	Never	Yes (webflow extension bundle)	Never

9. Shopify/Google UCP — From product.csv to Server-Side

9.1 The CSV Era and Why It's Ending

For over a decade, the e-commerce data pipeline looked like this:

```
Shopify Admin → Export CSV → Edit in Excel/Sheets → Upload to Google Merchant Center
                                        ↓
                                  Upload to CRM
                                        ↓
                                  Upload to CDP
                                        ↓
                                  Upload to consent platform
```

This worked when merchants had 50 products and 500 customers. It does not work when:

- Google requires real-time inventory and pricing (Merchant Center Next enforces server-side feeds)
- Shopify Customer Privacy API requires consent signals before data collection
- GA4 consent_mode v2 requires granular, real-time consent state (not a CSV column)
- GDPR Art. 17 (right to erasure) requires deletion propagated to all systems within 30 days — CSV workflows have no propagation mechanism
- Agentic AI needs machine-readable consent + entitlement state, not a spreadsheet

9.2 Shopify's Server-Side Shift

SHOPIFY CUSTOMER PRIVACY API + UCP

Shopify's Customer Privacy API (`shopify.customerPrivacy`) is now the canonical consent interface for Shopify storefronts. Combined with the User Consent Preferences (UCP) model:

CSV-ERA PATTERN	SERVER-SIDE REPLACEMENT	WHY
Export `customers.csv` with consent column	`customer.metafields` (crm_consent_*) via Admin API	Consent state must be real-time, not batch
Upload consent spreadsheet to OneTrust	Worker reads `user_claims` from Xano, pushes consent signals server-side	Consent changes propagate to all streams in < 1 second
Manual product feed CSV to Google Merchant	Shopify's Google & YouTube channel (server-side sync)	Google requires real-time price/availability; CSV feeds are deprecated for most categories
Export `products.csv` for Matrixify bulk edit	Shopify Admin API `productUpdate` mutation via GraphQL	Mutations are auditable, rollback-able, and respect access scopes
CSV customer import to CRM	`POST /sync/customers` (bearer-authed, logged)	Every sync is consent-checked, logged to `sync_log`, and tenant-isolated

SHOPIFY'S CONSENT SIGNAL FLOW (SERVER-SIDE)

```
┌──────────────────────────────────────────────────────────────┐
│  STOREFRONT (Client)                                         │
│                                                              │
│  shopify.customerPrivacy.setTrackingConsent({                │
│    analytics: true/false,                                    │
│    marketing: true/false,                                    │
│    preferences: true/false,                                  │
│    sale_of_data: true/false      ← CCPA                      │
│  })                                                          │
│       │                                                      │
│       ▼                                                      │
│  Shopify passes consent to checkout + pixels                 │
│       │                                                      │
│       ▼                                                      │
│  CRM Sync Worker receives via:                               │
│  ├─ Webhook (customer-update with consent metafields)        │
│  ├─ POST /auth/consent-sync (embed context)                  │
│  └─ Cron sync (reads consent from Xano user_claims)          │
│       │                                                      │
│       ▼                                                      │
│  Worker enforces consent before pushing to:                  │
│  GA4 / Adobe AEP / Salesforce / Klaviyo / HubSpot / etc.     │
└──────────────────────────────────────────────────────────────┘
```

9.3 Google's Server-Side Shift

GA4 MEASUREMENT PROTOCOL V2 + CONSENT MODE V2

Google's deprecation path is clear:

DEPRECATED	REPLACEMENT	DEADLINE
Universal Analytics (UA)	GA4	Completed (July 2024)
UA Measurement Protocol v1	GA4 Measurement Protocol v2	Completed
consent_mode v1 (2 signals)	consent_mode v2 (4 signals)	March 2024 (EEA), global enforcement ongoing
Client-side-only tracking	Server-side tagging (sGTM)	Recommended for consent compliance
Page-view conversion schema	Event-based conversion schema	GA4 native (no page-view conversions)
Product data CSV feeds	Google Content API / Merchant Center Next API	Enforced for real-time inventory

PAGE-VIEW CONVERSION SCHEMA → EVENT-BASED SCHEMA

This is the most impactful change for legacy apps that built around UA's page-view model:

```
UA (Legacy):
  Page View → Virtual Page View → Goal → Conversion
  /thank-you → pageview hit → destination goal → conversion counted

GA4 (Current):
  Event → Conversion Event → Key Event
  purchase → event with parameters → marked as key event → conversion counted
```

UA PAGE-VIEW PATTERN	GA4 EVENT EQUIVALENT	IMPACT ON LEGACY APPS
`/thank-you` destination goal	`purchase` event with `transaction_id`, `value`, `items[]`	Apps that track conversions by URL path break — no page-view goals in GA4
`/signup-complete` goal	`sign_up` event with `method` parameter	OneTrust/CMP integrations that fire on page load must fire events instead
Virtual pageview (`/vpv/funnel-step-3`)	Custom event `funnel_step` with `step_number: 3`	Matrixify CSV exports with vpv-based conversion data are meaningless in GA4
Session-based conversion window	Event-based attribution (data-driven)	CRM systems that import UA session data need to import GA4 event streams
Goal value (static)	Event value (dynamic, per-event `value` parameter)	CSV-imported static goal values don't exist — value is on each event

9.4 Impact on Legacy Applications

MATRIXIFY (FORMERLY EXCELIFY)

What it does: Bulk import/export Shopify data via CSV/Excel — products, customers, orders, metafields.

Legacy risk:

RISK	DESCRIPTION	CRM SYNC ALTERNATIVE
No consent enforcement	Matrixify CSV export dumps all customer data regardless of consent state	Worker checks `user_claims` consent before any data leaves Xano
No audit trail	Who exported what customer data, when? No log.	Every sync logged to `sync_log` with user, timestamp, record count
Stale data round-trips	Export → edit → re-import cycle can take hours/days — data drifts	Real-time webhook + 15-min cron — max staleness = 15 minutes
Credential in download	CSV files with customer emails, phones, addresses sitting in Downloads folder	PII hashed (SHA-256) before any external push; raw PII stays in Xano + Shopify
No GDPR deletion propagation	If a customer requests deletion, Matrixify CSVs in the wild still contain their data	GDPR redaction handler anonymizes across all systems; no CSVs to recall
Schema lock-in	Matrixify CSV schema is fixed to Shopify's export format — no consent fields, no GA4 event data	Worker schema is extensible — add fields to `CrmSiteConfig` interface

Migration path: Replace Matrixify customer exports with `GET /admin/shopify-customers` (bearer-authed, returns current data from Shopify Admin API). Replace Matrixify customer imports with `POST /sync/customers` (validates, consent-checks, logs). Product CSV operations remain in Matrixify (CRM Sync does not manage product data — that's PIM Sync's domain).

LEGACY CRMS (HUBSPOT CSV IMPORT, SALESFORCE DATA LOADER, ZOHO IMPORT)

What they do: Bulk CSV import of customer records into CRM contact databases.

Legacy risk:

RISK	DESCRIPTION	CRM SYNC ALTERNATIVE
Consent laundering	CSV import creates CRM contacts without consent verification — the CRM assumes consent was collected, but the CSV doesn't prove it	Every stream push checks `user_claims.consent_marketing` (or stream-specific consent) before sending. No consent = no push. Logged.
Duplicate identity	CSV imports create duplicate contacts (email case sensitivity, name variants)	Worker uses email as canonical identity key; upsert pattern prevents duplicates
No sync-back	CRM edits (sales rep updates a phone number) don't flow back to Shopify/Xano	Bidirectional: Webflow CMS webhook → Worker → Xano. CRM integrations can push changes back via Worker API
Orphaned records	Customer deleted in Shopify but still exists in CRM (no deletion propagation)	GDPR handler (`/gdpr/customer-redact`) propagates deletion to all connected streams
Flat file PII exposure	Customer CSVs emailed between teams, stored in shared drives	Zero CSV generation — all data flows through authenticated, encrypted API channels

Migration path: Replace CSV imports with connected streams (Section 3). Each CRM gets:

1. Config fields in `CrmSiteConfig` (`hubspot_enabled` , `hubspot_api_key` , etc.)
2. A `push{CRM}Event()` function in the Worker
3. A `{crm}_sync_log` table in Xano
4. Consent gate in the push chain

ONETRUST (AND LEGACY CMPS)

What it does: Cookie consent management platform. Manages consent banners, preference centers, and compliance reporting.

Legacy risk with CSV/page-view architecture:

RISK	DESCRIPTION	CRM SYNC ALTERNATIVE
Page-view consent model	OneTrust fires consent signals on page load — tied to UA's page-view hit model. GA4's event model means consent must be checked per-event, not per-page.	CRM Sync consent is event-level: every `pushToStream()` call checks consent state from `user_claims`. Consent travels with the data, not with the page.
Client-side only	OneTrust runs in the browser — if JS is blocked, consent isn't collected, but tracking may still fire via server-side tags	Worker enforces consent server-side. Even if client-side consent banner fails, the Worker defaults to `denied` for all signals. No consent = no data push.
Cookie-centric	OneTrust manages cookie categories (strictly necessary, performance, targeting, functional). GA4 consent_mode v2 uses different taxonomy (`ad_storage`, `analytics_storage`, `ad_user_data`, `ad_personalization`).	CRM Sync maps between taxonomies (see Section 2.2) and stores both representations in `consent_records` for audit.
No server-side state	OneTrust stores consent in cookies/localStorage — no server-side persistence accessible to the Worker	CRM Sync persists consent to Xano `user_claims` (server-side, authenticated) and `consent_records` (append-only audit log). Consent state survives cookie clearing.
Vendor lock-in	OneTrust consent categories are proprietary. Migrating to another CMP requires re-mapping all categories.	CRM Sync consent model is standards-based (GA4 consent_mode v2 + GDPR legal basis). The Worker is CMP-agnostic — it reads consent signals, not OneTrust categories.
No agentic readability	OneTrust's consent state is in browser cookies — an AI agent cannot read cookies to verify consent before processing a payment	CRM Sync consent is in Xano API (`user_claims`) — an agent calls the API, gets machine-readable consent state, and makes a verifiable decision

Migration path: OneTrust (or any CMP) can coexist with CRM Sync. The CMP manages the UI banner; CRM Sync manages the server-side consent state:

```
OneTrust Banner (client)
  │
  ├─ Sets cookies (OneTrust's domain)
  ├─ Fires gtag('consent', 'update', {...})  ← GA4 consent_mode v2
  │
  └─ POST /auth/consent-sync (CRM Sync Worker)
       │
       ├─ Writes to user_claims (Xano — server-side truth)
       ├─ Writes to consent_records (Xano — audit log)
       └─ Maps CMP categories → GA4 signals → stream-specific consent
```

CRM Sync doesn't replace the CMP banner — it replaces the **server-side consent enforcement** that OneTrust doesn't provide. OneTrust tells the browser what's allowed. CRM Sync tells the server what's allowed.

PAGE-VIEW CONVERSION SCHEMA — WHAT DIES

The page-view conversion schema affects every tool that reports on conversions:

TOOL	PAGE-VIEW DEPENDENCY	WHAT BREAKS	MIGRATION
Google Ads	UA imported goals as conversion actions	GA4 key events replace goals — must re-link	Re-configure conversion actions in Google Ads from GA4 key events
Google Analytics reports	UA goal funnel visualization (page-path based)	No equivalent in GA4 — funnels are event-based	Build GA4 funnel explorations using custom events
OneTrust analytics	OneTrust reports consent by page (page-view correlation)	GA4 doesn't associate consent with pages — consent is a user-level state	Use CRM Sync consent_records for per-user, per-event consent reporting
Matrixify reports	Export UA goal completions as CSV column	GA4 has no goal completions — has key event counts per event name	Use GA4 Data API or CRM Sync sync_logs for conversion data
HubSpot attribution	HubSpot reads UA source/medium from page-view hit	GA4 attribution is data-driven (cross-session, cross-device)	Use HubSpot's native GA4 integration or push attribution via Worker
Salesforce Pardot	Pardot tracks page views for lead scoring	GA4 events replace page views for scoring signals	Push CRM events (tag mutations, consent changes) as Salesforce activities via Worker
Custom dashboards	Looker/Tableau queries UA goal data via BigQuery export	UA BigQuery schema ≠ GA4 BigQuery schema — queries break	Rewrite queries for GA4 event schema + supplement with CRM Sync sync_logs

9.5 The UCP Security/Privacy Model — Server-Side Consent as Infrastructure

WHY SERVER-SIDE CONSENT CHANGES EVERYTHING

In the CSV/page-view era, consent was a **client-side suggestion**. The browser said "user consented to analytics" and every downstream system trusted that signal. But:

1. **Client-side consent can be spoofed.** A browser extension, ad blocker, or malicious script can forge consent signals. Server-side consent verification (checking `user_claims` in Xano) cannot be spoofed by the client.
2. **Client-side consent doesn't persist.** User clears cookies → consent state is lost → system defaults to either "re-ask" (friction) or "assume granted" (illegal under GDPR). Server-side consent in Xano persists indefinitely, tied to the authenticated user identity.
3. **Client-side consent can't propagate.** OneTrust sets a cookie. How does Salesforce know about it? CSV export? Manual sync? CRM Sync propagates consent changes to all connected streams within the same request cycle — consent change → check all streams → push updates → log results.
4. **Client-side consent isn't auditable for agents.** An AI agent processing a payment needs to verify: "Does this user consent to marketing data processing?" A cookie is not an API. `GET /auth/me` returns machine-readable consent state from Xano — the agent can make a provable decision.

UCP PRIVACY GUARANTEES (SERVER-SIDE)

GUARANTEE	HOW CRM SYNC ENFORCES IT
Consent before collection	`shopify.customerPrivacy.setTrackingConsent()` fires before any pixel/tag; Worker defaults to `denied` if consent unknown
Consent before processing	Every `pushToStream()` reads `user_claims` consent state before sending data
Consent before sharing	Each stream has its own consent requirement (see Section 3.3); consent is checked per-stream, not globally
Right to withdraw	Consent toggle in UCP dashboard → `POST /auth/consent-sync` → immediate propagation to all streams
Right to erasure	`POST /gdpr/customer-redact` → anonymize in Xano, delete from Webflow CMS, notify all streams
Right to access	UCP dashboard shows consent history, sync history, and which systems have user data
Data minimization	Only consented data categories are pushed; PII hashed (SHA-256) for analytics streams
Purpose limitation	Each stream declares its purpose (analytics, marketing, CRM); consent is purpose-specific
Records of processing	`consent_records` (append-only) + per-stream `sync_log` = complete Art. 30 record

9.6 Legacy App Migration Decision Matrix

For organizations evaluating which legacy tools to replace vs. keep:

LEGACY TOOL	KEEP / REPLACE / AUGMENT	RATIONALE
Matrixify	**Replace** (for customer data)	No consent enforcement, no audit trail, PII in CSVs. Keep for product bulk ops only (PIM Sync domain).
OneTrust	**Augment**	Keep the banner UI. Replace server-side consent enforcement with CRM Sync. OneTrust manages the UX; CRM Sync manages the truth.
HubSpot CSV Import	**Replace**	Use connected stream via Worker. Consent-gated, logged, deduplicated.
Salesforce Data Loader	**Replace**	Use connected stream. No more flat-file PII. Consent checked before every push.
Klaviyo CSV List Import	**Replace**	Use connected stream. Email consent verified per subscriber before push.
Google Merchant CSV Feed	**Replace**	Use Shopify's Google channel (server-side). Or Content API via Worker for custom feeds.
UA Goal-Based Reporting	**Replace**	Rebuild on GA4 key events. No migration path — UA goals are structurally incompatible with GA4.
Looker/Tableau UA Queries	**Rewrite**	GA4 BigQuery schema is different. Supplement with CRM Sync sync_logs for consent + stream data.
Shopify Customer CSV Export	**Replace**	Use `GET /admin/shopify-customers` (bearer-authed). Or Shopify Admin API directly. No PII in downloads.

9.7 Timeline Pressure

DEADLINE	WHAT HAPPENS	LEGACY IMPACT
Already passed (July 2024)	UA stopped processing data	UA-shaped CSV exports contain no new data. Historical only.
Already passed (March 2024)	consent_mode v2 required in EEA for Google Ads	Ads campaigns in EEA without v2 signals lose remarketing + conversions
Ongoing (2025-2026)	Google Merchant Center Next enforced	CSV product feeds deprecated for most categories; server-side feeds required
Ongoing (2025-2026)	Shopify Customer Privacy API required for new apps	Apps that don't implement `shopify.customerPrivacy` will be rejected from app store
2026 H2 (projected)	GDPR enforcement actions increasing	Regulators targeting consent laundering (importing contacts without verifiable consent)
2026-2027	AI/Agentic payment regulations emerging	Payment processors requiring machine-readable consent verification for AI-initiated transactions

Organizations still using CSV workflows for customer data are accumulating compliance debt with each passing month. The page-view conversion schema is already dead — UA stopped processing in July 2024. The question is not whether to migrate, but how much historical data and process debt to carry forward.

---

10. Shopify App Pivot — Partner Dashboard → Dev Dashboard

10.1 What Changed and Why

Shopify has been migrating app management from the **Partner Dashboard** (partners.shopify.com) to the **Dev Dashboard** (dev.shopify.com). This is not a

cosmetic rebrand — it reflects fundamental changes to how apps are created, configured, authenticated, and reviewed. Apps built against the old Partner Dashboard patterns will fail submission under the new requirements.

```
PARTNER DASHBOARD (Legacy)                DEV DASHBOARD (Current)
_________________________                 _____________________

App created in web UI                     App created via CLI or dev.shopify.com
Scopes configured in dashboard            Scopes declared in shopify.app.toml
Redirect URLs in dashboard form           redirect_urls in [auth] config
Extensions managed in dashboard           Extensions managed via CLI
OAuth: non-expiring offline tokens        OAuth: expiring tokens (60-min access, 90-day refresh
GDPR webhooks: optional checkbox          GDPR webhooks: mandatory compliance_topics
API version: implicit                     API version: explicit in webhooks.api_version
Review: manual, opaque timeline           Review: AI-assisted self-review + structured submissi
Protected customer data: blanket scopes   Protected customer data: field-level access requests
Billing: REST API                         Billing: GraphQL API (App Subscription)
```

10.2 Critical Pivot Changes

A. AUTHENTICATION — EXPIRING TOKENS (MANDATORY SINCE APRIL 1, 2026)

Partner Dashboard era: Apps received non-expiring offline access tokens (`shpat_*`). Store the token once, use forever.

Dev Dashboard era: New public apps MUST use expiring tokens:

- Access token (`shpua_*`): 60-minute lifetime
- Refresh token (`shprt_*`): 90-day lifetime
- Token exchange must send `expiring=1` parameter
- App must refresh proactively (before expiry, not after 401)
- Both `access_token` and `refresh_token` update on every refresh

CHECK	LEGACY PATTERN	REQUIRED PATTERN	CRM SYNC STATUS
Token type	`shpat_*` (never expires)	`shpua_*` (60 min) + `shprt_*` (90 day)	`refreshShopifyTokenIfNeeded()` implemented
Refresh trigger	After 401 error	Before expiry (proactive)	Proactive check on every request
Storage	Token in env var or config	Token + refresh + expiry timestamp in KV	`tenant:{shop}:config` stores all three
Failure handling	None (token never expires)	Retry refresh, alert on failure	Fallback 401 handler + logging

B. PROTECTED CUSTOMER DATA — FIELD-LEVEL ACCESS

Partner Dashboard era: Request `read_customers` scope → get all customer fields.

Dev Dashboard era: Three access levels with field-level granularity:

LEVEL	ACCESS	FIELDS	REQUIREMENT
Level 0	No protected data	Public fields only (orders, products)	Default
Level 1	Basic customer data	`read_customer_name`, `read_customer_email`	Justify in submission
Level 2	Full customer data	+ `read_customer_phone`, `read_customer_address`	Data protection details required

CRM Sync impact: CRM Sync requires Level 1 minimum (`read_customer_name`, `read_customer_email`) for identity resolution. Level 2 if syncing phone/address to CDPs.

CHECK	WHAT TO VERIFY
Access level declared	Level 1 or 2 requested in Partner Dashboard → Protected Customer Data section
Field-level scopes	`read_customer_name`, `read_customer_email` at minimum
Null handling	Code handles `null` for unapproved/redacted fields without crashing
Dev store caveat	Dev stores bypass field scoping — must test on non-dev store

C. CONFIGURATION AS CODE — SHOPIFY.APP.TOML

Partner Dashboard era: App config in web forms. No version control. No diffing. No PR review.

Dev Dashboard era: `shopify.app.toml` is the single source of truth:

```
# shopify.app.toml — version-controlled, PR-reviewable, diffable

name = "CRM Sync"

client_id = "0e57977712f8a9d270a602848ff95308"

application_url = "https://hx-crm-sync.yoonsunlee150.workers.dev"

embedded = true


[auth]

redirect_urls = [

  "https://hx-crm-sync.yoonsunlee150.workers.dev/auth/callback"

]


[webhooks]

api_version = "2026-07"


[compliance_webhooks]

customer_deletion_url = "https://hx-crm-sync.yoonsunlee150.workers.dev/gdpr/customer-redact"

customer_data_request_url = "https://hx-crm-sync.yoonsunlee150.workers.dev/gdpr/data-request"

shop_deletion_url = "https://hx-crm-sync.yoonsunlee150.workers.dev/gdpr/shop-redact"


[access_scopes]

scopes = "read_customers,write_customers,read_orders"

use_legacy_install_flow = false
```

CHECK	WHAT TO VERIFY
`shopify.app.toml` exists	File present in repo root
`client_id` matches	Same as Dev Dashboard app
`embedded = true`	If app renders in Shopify Admin
`use_legacy_install_flow = false`	Not using deprecated OAuth flow
`compliance_webhooks` declared	All three GDPR endpoints configured
`api_version` current	Not deprecated or sunset (2025-04 or later)
`redirect_urls` correct	Points to Worker callback, not localhost
`scopes` minimal	Only scopes the app actually uses

D. EXTENSIONS — CLI-MANAGED, NOT DASHBOARD-MANAGED

Partner Dashboard era: Extensions created and configured in web UI. Bundle uploaded manually.

Dev Dashboard era: Extensions managed via Shopify CLI:

```
shopify app generate extension     # Create extension scaffold
shopify app dev                    # Local development server
shopify app deploy                 # Deploy extensions + update config
```

CHECK	WHAT TO VERIFY
Extensions in repo	`extensions/` directory with extension config
CLI version	`shopify version` >= 3.84.1
No dashboard-managed extensions	All extensions have local config files
Deploy via CLI	`shopify app deploy` (not manual dashboard upload)

E. GDPR COMPLIANCE WEBHOOKS — MANDATORY

Partner Dashboard era: GDPR webhooks were an opt-in checkbox. Many apps never implemented them.

Dev Dashboard era: `compliance_topics` must be declared. Handlers must:

CHECK	REQUIREMENT
`customers/data_request`	Returns all stored data for a customer (GDPR Art. 15)
`customers/redact`	Deletes/anonymizes customer PII (GDPR Art. 17)
`shop/redact`	Deletes all data 48h after app uninstall
HMAC validation	All handlers verify `X-Shopify-Hmac-SHA256`
Response code	All handlers return 200-series
Content-Type	Accept `application/json` POST body

CRM Sync status: All three handlers implemented with HMAC verification (see Security Audit, routes #43-45).

F. BILLING — GRAPHQL APPSUBSCRIPTION API

Partner Dashboard era: REST Billing API. Simple recurring charges.

Dev Dashboard era: GraphQL `appSubscriptionCreate` mutation with:

CHECK	WHAT TO VERIFY
Shopify Billing API used	No external payment processing (PayPal, Stripe direct)
`test: true` for dev	Billing tested on dev store with test flag
`test: false` for production	Test flag removed before submission
Upgrade/downgrade	Merchant can change plan without reinstalling
Enterprise pricing	Described in "Description of additional charges"

G. APP REVIEW — AI SELF-REVIEW + STRUCTURED SUBMISSION

Partner Dashboard era: Submit app → wait weeks → opaque feedback.

Dev Dashboard era (April 2026+):

1. AI-assisted self-review flags common issues before submission
2. Structured submission form with specific sections for each requirement
3. Test credentials must be provided and functional
4. Demo screencast required (English or English subtitles)
5. Emergency contact (email + phone) required

10.3 CRM Sync — Current Compliance Status

#	REQUIREMENT	STATUS	NOTES
1	App in Dev Dashboard	✓	client_id `0e57977712f8a9d270a602848ff95308`
2	`shopify.app.toml` exists	✓	In repo root
3	Expiring tokens implemented	✓	`refreshShopifyTokenIfNeeded()` with proactive refresh
4	Protected customer data level		Need to request Level 1 in Partner Dashboard
5	Null field handling	✓	GraphQL queries handle optional fields
6	GDPR webhooks implemented	✓	Routes #43-45, HMAC verified
7	GDPR webhooks in `compliance_webhooks`		Verify in `shopify.app.toml`
8	Extensions via CLI	✓	`extensions/crm-auth/` managed locally
9	Billing via GraphQL		Not yet implemented
10	Privacy policy URL	✓	`docs/privacy.html` deployed
11	Scopes minimal		Audit needed
12	`use_legacy_install_flow = false`		Verify in toml
13	API version current		Verify `webhooks.api_version`
14	App listing complete		Screenshots, demo video, test creds needed
15	Emergency contact provided		Need email + phone

10.4 Downloadable Markdown Checklist

The full Shopify App pivot checklist is available as a standalone markdown file for download and tracking:

File: `docs/shopify-app-checklist.llm.md`

This file can be:

1. **Fed to any LLM** (Claude, GPT, Gemini) along with the codebase for automated compliance audit
2. **Used as a PR checklist** — paste into a GitHub PR description for team review
3. **Tracked in project management** — each numbered item maps to a task

QUICK REFERENCE — CHECKLIST SECTIONS

SECTION	ITEMS	FOCUS
1. Dev Dashboard & Config	1.1–1.11	`shopify.app.toml`, CLI version, extensions
2. Authentication & Tokens	2.1–2.14	Expiring tokens, OAuth flow, session management
3. Protected Customer Data	3.1–3.6	Field-level access, null handling, dev store caveat
4. Data Security	4.1–4.6	Encryption, HMAC, secrets management
5. GDPR / Privacy	5.1–5.10	Compliance webhooks, privacy policy, data minimization
6. App Store Listing	6.1–6.14	Screenshots, demo video, contact info
7. Billing	7.1–7.5	GraphQL billing, test mode, plan changes
8. App Functionality	8.1–8.10	Checkout, rate limits, idempotency
9. Webhooks & Sync	9.1–9.5	GraphQL registration, HMAC, response time
10. Post-Launch	10.1–10.5	Version currency, monitoring, scope changes

KEY DATES

DATE	CHANGE	IMPACT
Feb 2025	Scopes reviewed for necessity on every submission	Remove unused scopes before submitting
Dec 2025	Protected customer data scopes enforced for web pixels	Pixels that access customer data need field-level approval
Apr 1, 2026	**Expiring offline tokens mandatory for new public apps**	Apps with non-expiring tokens will be rejected
Mar 2026	RBAC for partner orgs; clearer image standards	Multi-user partner orgs need role setup
Apr 2026	New submission experience with AI self-review	Pre-submission automated checks

DOWNLOAD & USE

```
# Download the checklist
curl -O https://raw.githubusercontent.com/persephonepunch/crm-sync/master/docs/shopify-app-chec


# Feed to Claude for automated audit
cat shopify-app-checklist.llm.md src/index.ts | claude "Audit this app against the checklist"


# Or use as a GitHub PR template
cat shopify-app-checklist.llm.md >> .github/PULL_REQUEST_TEMPLATE/app-submission.md
```

10.5 Partner Dashboard → Dev Dashboard Migration Checklist

For apps migrating from Partner Dashboard to Dev Dashboard:

- ☐ **Create app in Dev Dashboard** (dev.shopify.com) or migrate existing app
- ☐ **Generate** `shopify.app.toml` via `shopify app config push` or create manually
- ☐ **Move scopes** from dashboard form to `[access_scopes]` in toml
- ☐ **Move redirect URLs** from dashboard form to `[auth].redirect_urls` in toml

- [ ] **Implement expiring tokens** — replace `shpat_*` with `shpua_*` + `shprt_*` flow
- [ ] **Add proactive token refresh** — check expiry before every API call
- [ ] **Implement all three GDPR webhooks** — data request, customer redact, shop redact
- [ ] **Add HMAC validation** to all webhook handlers
- [ ] **Declare** `compliance_webhooks` in toml
- [ ] **Request protected customer data** access in Partner Dashboard (Level 1 or 2)
- [ ] **Test on non-dev store** — dev stores bypass field-level scoping
- [ ] **Handle null fields** — unapproved/redacted fields return null
- [ ] **Switch billing to GraphQL** `appSubscriptionCreate` mutation
- [ ] **Remove** `test: true` from billing before submission
- [ ] **Install Shopify CLI >= 3.84.1** — `npm install -g @shopify/cli`
- [ ] **Move extensions to CLI management** — `shopify app generate extension`
- [ ] **Deploy via CLI** — `shopify app deploy` (not dashboard upload)
- [ ] **Set** `use_legacy_install_flow = false` in toml
- [ ] **Set** `webhooks.api_version` to current supported version (2025-04+)
- [ ] **Create app listing** — icon, screenshots, demo video, test credentials
- [ ] **Add emergency contact** — email + phone
- [ ] **Create privacy policy** — link from app listing
- [ ] **Run AI self-review** — fix flagged issues before human review
- [ ] **Verify Chrome incognito** — app works without existing cookies/sessions

---

11. Architecture Comparison — Distributed Decentralized Build vs. Monolithic SSR

11.1 The Two Architectures

This section compares two complete application architectures for building authenticated, multi-tenant SaaS products that handle customer PII, consent state, and payment entitlements:

Architecture A (CRM Sync — Distributed Decentralized Build): Webflow/TypeScript/Vite Components + Cloudflare Workers/Functions + Xano API + TLS Auth

Architecture B (Conventional Full-Stack — Monolithic SSR): Next.js RSC + Prisma/Drizzle ORM + File System API + CSR/SSR Hydration

```
ARCHITECTURE A (Distributed)                ARCHITECTURE B (Monolithic)
_____________________________________       _____________________________________

UI:      Webflow Components (Vite)          UI:      Next.js React Server Components
Logic:   Cloudflare Workers (V8 isolate)    Logic:   Next.js API routes + Server Actions
Data:    Xano API (managed PostgreSQL)      Data:    Prisma/Drizzle → self-managed DB
Auth:    Worker JWT + OAuth + HMAC + TLS    Auth:    NextAuth/Auth.js + session cookies
State:   Cloudflare KV (encrypted)          State:   File system / Redis / DB sessions
Deploy:  Wrangler (edge, immutable)         Deploy:  Vercel/Node (origin, mutable)
Deps:    0 runtime npm packages            Deps:    200-800+ npm packages
```

11.2 Layer-by-Layer Comparison

UI LAYER: WEBFLOW/VITE COMPONENTS VS. NEXT.JS RSC

DIMENSION	WEBFLOW + TYPESCRIPT + VITE	NEXT.JS RSC
Rendering model	Pre-compiled static components; hydration-free in Designer sandbox	Server Components (RSC) + Client Components; partial hydration via React runtime
Bundle size	~12KB (CRM Auth extension bundle)	80-300KB+ baseline (React runtime + RSC payload + client components)
Build tool	Vite (ESBuild-based, sub-second HMR)	Next.js compiler (Turbopack/Webpack — slower, more complex)
Component model	TypeScript → compile → bundle.zip → upload to Webflow	`.tsx` files in `app/` directory → built by Next.js → deployed as Node server
Runtime dependencies	Zero npm deps in production bundle	React, ReactDOM, Next.js runtime, RSC wire format parser
Design system	Webflow Semantic Design (variables, classes, conditional visibility)	CSS Modules / Tailwind / styled-components (code-managed)
CMS integration	Native Webflow CMS API (structured content)	Headless CMS via fetch or SDK (additional dependency)

Security implication: RSC introduces a new attack surface — the server/client component boundary. A `"use server"` directive that accidentally exposes a function allows direct invocation from the client. Webflow components run in a Designer sandbox with no server execution context — the boundary is physical (browser → Worker API), not a directive annotation.

COMPUTE LAYER: CLOUDFLARE WORKERS VS. NEXT.JS API ROUTES

DIMENSION	CLOUDFLARE WORKERS	NEXT.JS API ROUTES / SERVER ACTIONS
Runtime	V8 isolate — no Node.js, no `fs`, no `child_process`	Node.js — full access to filesystem, processes, network
Isolation	Each request runs in its own V8 isolate — zero shared memory	Shared Node.js process — requests share memory, event loop, global state
Cold start	~0ms (V8 isolates are pre-warmed at edge)	250ms-3s (Node.js process startup, especially with large dependency trees)
File system access	None — impossible to read/write files	Full `fs` access — read/write any file the process can reach
Process execution	None — `child_process` doesn't exist	`exec()`, `spawn()` available — command injection surface
Network	`fetch()` only — no raw socket, no DNS rebinding	Full `net` module — raw TCP, UDP, DNS resolution
Concurrency model	Request-level isolation (like a new process per request)	Event loop shared across all concurrent requests
Global state	None between requests (V8 isolate disposed after response)	`global` / `process.env` persist across requests — state leaks possible
Max execution	30s (Workers paid plan)	Unlimited (Vercel: 10s-300s depending on plan; self-hosted: unlimited)
Location	300+ edge locations worldwide	1 region (Vercel: edge functions available but limited)

Security implication: The Worker's restricted runtime is a **security feature, not a limitation**. No `fs` means no path traversal attacks. No `child_process` means no command injection. No shared memory means no cross-request data leaks. No `eval()` means no code injection. These attack classes are **structurally impossible** in the V8 isolate model — they don't need to be mitigated because they can't exist.

DATA LAYER: XANO API VS. PRISMA/DRIZZLE ORM

DIMENSION	XANO API (DOCKER/KUBERNETES MANAGED)	PRISMA / DRIZZLE ORM
Database access	API-only — no SQL from application code	Direct SQL generation — ORM constructs and executes queries
SQL injection	Impossible — application never writes SQL	Possible — raw queries (`prisma.$queryRaw` , `drizzle.execute`) bypass ORM protection
Schema management	Xano dashboard — schema changes are UI operations	Migration files — `prisma migrate` / `drizzle-kit push` — code + file system operations
Connection management	Xano handles connection pooling internally	Application manages connection pool (PgBouncer, Prisma Accelerate, etc.)
Connection string	No connection string in application — API key only	`DATABASE_URL` with credentials in env var — compromise = full DB access
Multi-tenancy	Row-level or table-level via API endpoint design	Schema-level or row-level — must implement manually in ORM queries
Backups	Xano-managed (automated)	Self-managed (cron + pg_dump, or cloud provider snapshots)
Scaling	Xano auto-scales (Docker/K8s under the hood)	Manual — configure replicas, read replicas, connection limits
Audit trail	Built into Xano (request logs, table history)	Must implement manually (audit trigger, event sourcing, or middleware)

Security implication: With Prisma/Drizzle, the application server has **direct database credentials**. A Server Action vulnerability, SSRF, or environment variable leak exposes the full connection string — and with it, `SELECT * FROM users`. With Xano, the application has an **API key** that only grants access to exposed API endpoints — not raw table access. The blast radius of a credential leak is fundamentally different:

```
Prisma credential leak:
  DATABASE_URL="postgresql://user:pass@host:5432/db"
  → Attacker: SELECT * FROM users; DROP TABLE consent_records;
  → Full read/write/delete on every table

Xano API key leak:
  XANO_API_KEY="xano_abc123"
  → Attacker: can call exposed API endpoints only
  → Cannot run arbitrary SQL
  → Cannot access tables without endpoints
  → Cannot DROP, ALTER, or TRUNCATE anything
  → Rate-limited by Xano
```

AUTH LAYER: WORKER JWT + TLS VS. NEXTAUTH/AUTH.JS

DIMENSION	WORKER JWT + OAUTH + HMAC + TLS	NEXTAUTH / AUTH.JS
Session storage	JWT in httpOnly cookie, signed with `JWT_SECRET` via Worker `crypto.subtle`	Session in database/file/JWT — configurable, default often insecure
Token signing	Web Crypto API (hardware-backed on Cloudflare edge)	Node.js `crypto` module (software)
OAuth implementation	Hand-rolled in Worker — minimal, auditable, zero deps	NextAuth adapter — large dependency tree, opaque middleware
CSRF protection	State nonce in KV (single-use, TTL)	NextAuth built-in (but configurable → misconfigurable)
Webhook auth	HMAC-SHA256 per handler (explicit verification)	No built-in webhook verification — manual implementation
Multi-provider	Google, Shopify, Webflow OAuth — each provider is a `fetch()` call	Provider adapters — each adapter is an npm package with its own deps
TLS	Cloudflare-managed (automatic, edge-terminated)	Reverse proxy or platform-managed (Vercel, AWS)
Session fixation	Impossible — JWT is signed, not stored server-side	Possible with database sessions if not properly rotated
Auth middleware complexity	~30 lines (`verifyBearerToken` , `verifyAdminKey` , JWT decode)	NextAuth middleware — hundreds of lines of config, callbacks, adapters

Security implication: NextAuth's flexibility is its risk. The adapter pattern means auth behavior depends on which database adapter, which session strategy, and which callback configuration the developer chose. Misconfiguration (leaving `NEXTAUTH_SECRET` as default, using `jwt` strategy with database adapter, forgetting to set `secureCookie: true`) creates vulnerabilities that static analysis can't catch because they're configuration errors, not code errors. The Worker's auth is code — it either verifies the HMAC or it doesn't. No configuration to misconfigure.

STATE LAYER: CLOUDFLARE KV VS. FILE SYSTEM / REDIS

DIMENSION	CLOUDFLARE KV	FILE SYSTEM API / REDIS / DB SESSIONS
Encryption at rest	Always (Cloudflare-managed)	File system: never by default. Redis: optional (rarely enabled). DB: depends on provider.
TTL support	Native (per-key expiry)	File: none (manual cleanup). Redis: native. DB: manual column + cron.
Access control	KV namespace bound to specific Worker — other Workers cannot read	File system: any process with OS permissions. Redis: AUTH command (optional). DB: connection credentials.
Global distribution	Replicated to 300+ edge locations	File: single server. Redis: Cluster or Sentinel (manual). DB: read replicas (manual).
Path traversal risk	Impossible — keys are strings, not file paths	File system API: `../../etc/passwd` attacks if input not sanitized
Concurrent write safety	Eventual consistency (last write wins, globally)	File: race conditions without locking. Redis: atomic ops available. DB: transactions.
Secrets in state	Encrypted, never visible in dashboard	File: plaintext on disk. Redis: plaintext in memory (RDB dump to disk).

Security implication: The File System API in Next.js/Node.js applications is a recurring source of vulnerabilities. Server Actions that accept file paths, image upload handlers that write to disk, cache files that store session data — all create path traversal and local file inclusion (LFI) attack surfaces. Cloudflare Workers have **no file system** — this entire category of vulnerability is eliminated by the runtime.

11.3 Hydration Security Risks

CSR/SSR HYDRATION — THE SERIALIZATION BOUNDARY PROBLEM

Next.js RSC serializes component trees from server to client. This serialization is the **most novel attack surface in modern React**:

```
Server Component renders → RSC payload (JSON-like wire format) → Client deserializes → Hydratic


Risk points:
1. Server Component accidentally includes server-only data in render output
2. RSC payload includes props that were meant for server-only children
3. Client Component receives hydration data that contains secrets/tokens
4. Hydration mismatch → client re-renders with different (potentially exposed) data
```

HYDRATION RISK	DESCRIPTION	CRM SYNC (NO HYDRATION)
Props serialization leak	Server Component passes DB query results as props → serialized to client → visible in page source	No serialization — Worker returns JSON via `fetch()`. Client renders from API response only.
`"use server"` function exposure	Server Action becomes a callable endpoint. If it accepts user input without validation, it's an RCE vector.	No server actions — all mutations are explicit `POST` requests to Worker endpoints with auth.
Hydration mismatch XSS	Server renders sanitized HTML. Client hydration renders unsanitized user input → XSS.	No hydration — Webflow components render once from compiled bundle. No server/client divergence.
RSC payload injection	Attacker injects malicious data into RSC wire format during transit	No RSC payload — Worker responses are plain JSON with `Content-Type: application/json`.
Environment variable leak via RSC	`process.env.SECRET` accessible in Server Component → accidentally rendered → serialized to client	Workers use `env` parameter (per-request, isolated). No `process.env`. Cannot accidentally render secrets.
Streaming SSR timing attack	Streaming RSC reveals component render order → attacker infers conditional logic (auth checks, feature flags)	No streaming — Worker returns complete response. No partial render information leaks.

THE `"USE SERVER"` PROBLEM

Next.js Server Actions are functions annotated with `"use server"` that the client can invoke directly:

```
// Next.js Server Action — this becomes a POST endpoint automatically
"use server"
async function updateProfile(formData: FormData) {
  const email = formData.get("email");
  await db.user.update({ where: { id: session.userId }, data: { email } });
}
```

What can go wrong:

RISK	DESCRIPTION	WHY WORKERS DON'T HAVE THIS
Implicit endpoint	`"use server"` creates a POST endpoint. Developer may not realize it's network-callable.	Every Worker endpoint is explicit — you write `if (url.pathname === "/auth/profile")`. No implicit endpoints.
No auth by default	Server Actions don't require authentication unless the developer adds it.	Worker routes have `verifyBearerToken()` or `verifyJWT()` — auth is in the route handler, not a decorator.
Input validation gap	`FormData` arrives unvalidated. The ORM may sanitize SQL, but business logic validation is the developer's job.	Worker handlers parse JSON body and validate before passing to Xano API. Xano validates again at schema level.
Closure capture	Server Action closures can capture server-side variables and accidentally serialize them to the client.	No closures cross the network boundary. Request → Worker → Response. Data is explicit.
Enumeration	Each Server Action has a predictable endpoint ID. Attacker can enumerate and call actions they shouldn't have access to.	Worker routes are explicitly listed in the router. No auto-generated endpoint IDs.

11.4 Dependency Chain Risk

NPM SUPPLY CHAIN — 0 VS. 800+

```
CRM SYNC (Architecture A)                NEXT.JS APP (Architecture B)
_________________________                ___________________________

Runtime deps: 0                          Runtime deps: 200-800+

                                         ├─ next (core)
Worker uses:                             ├─ react, react-dom
├─ fetch()        (V8 built-in)          ├─ @prisma/client (or drizzle-orm)
├─ crypto.subtle  (V8 built-in)          ├─ next-auth (+ adapters)
├─ Request/Response (V8 built-in)        ├─ zod (validation)
├─ URL, Headers   (V8 built-in)          ├─ bcrypt / argon2
├─ TextEncoder    (V8 built-in)          ├─ jsonwebtoken
├─ KV bindings    (Cloudflare runtime)   ├─ cookie / express-session
└─ (nothing else)                        ├─ @tanstack/query
                                         ├─ axios / ky
                                         ├─ ... (200+ transitive deps)
                                         └─ Each dep = supply chain trust point
```

RISK	0 RUNTIME DEPS (WORKERS)	800+ DEPS (NEXT.JS + PRISMA)
Malicious package	Impossible — no packages to compromise	Any of 800+ packages could be compromised (xz-utils, event-stream, ua-parser-js precedent)
Typosquatting	N/A	`npm install prisma` vs `npm install prism` — one letter = malicious package
Abandoned package	N/A	Unmaintained dep with known CVE stays in tree until manually removed
Prototype pollution	V8 isolate — no `Object.prototype` mutation across requests	Shared Node.js process — prototype pollution affects all concurrent requests
ReDoS	Possible but limited to 30s Worker timeout	No timeout on Node.js — ReDoS can exhaust server resources indefinitely
Install scripts	N/A	`postinstall` scripts run arbitrary code during `npm install`
Audit surface	1 file (~7,200 lines) — human-auditable	Hundreds of files across node_modules — impossible to manually audit
Lock file integrity	N/A	`package-lock.json` must be verified — integrity hashes can be tampered
SBOM generation	Trivial (no deps = empty SBOM)	Complex — must enumerate all transitive deps with versions and licenses

11.5 Deployment Model Risk

DIMENSION	WRANGLER DEPLOY (WORKERS)	VERCEL / NODE.JS DEPLOY (NEXT.JS)
Artifact	Single JS file (~300KB compiled)	Docker image or Vercel build artifact (100MB-2GB)
Immutability	Each deploy creates immutable version — previous versions retained	Mutable deploys — previous version overwritten (unless Vercel preview)
Rollback	Deploy previous version forward (or restore KV config)	Re-deploy from git commit or Vercel rollback (not always instant)
Secrets	`wrangler secret put` — encrypted, never in code	`.env.local`, Vercel env vars, or secrets manager — multiple places to check
Build reproducibility	TypeScript → single file → deploy. Deterministic.	`npm ci` → build → bundle → deploy. Non-deterministic (npm registry, build cache, node version).
Preview environments	Wrangler preview (optional)	Vercel preview per PR (automatic — exposes preview URLs that may contain secrets)
Origin server	None — Worker runs at edge, no origin to attack	Node.js server (or serverless function with cold start) — origin exists and is attackable
DDoS surface	Cloudflare edge absorbs DDoS — Worker only sees valid requests	Origin server receives all traffic unless behind CDN/WAF
Config drift	Config in KV — same KV across all edge locations	Config in env vars — can diverge between environments

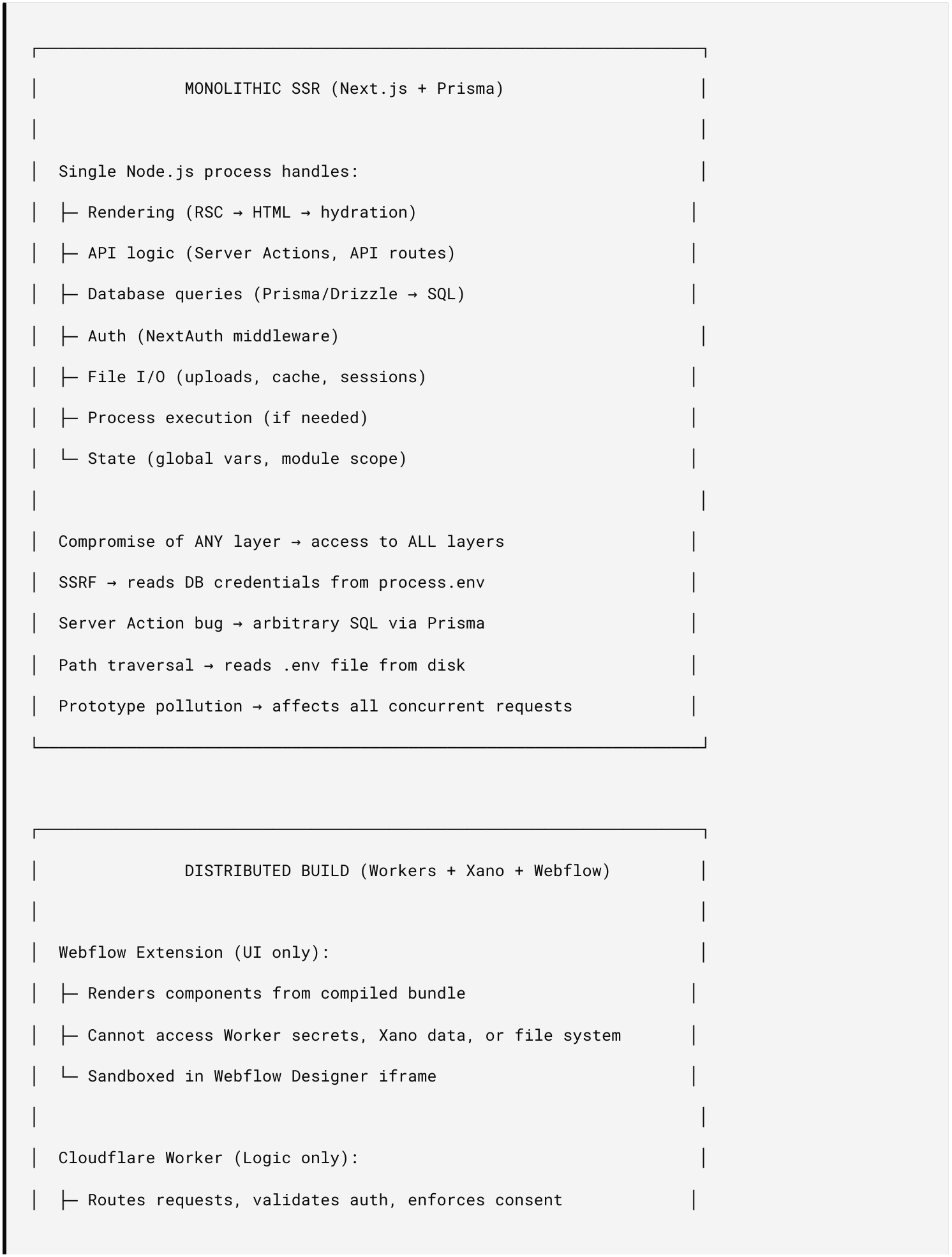
11.6 Specific CVE Classes Eliminated by Distributed Architecture

CVE CLASS	OWASP CATEGORY	NEXT.JS + PRISMA RISK	CRM SYNC (WORKERS + XANO)
SQL Injection	A03:2021	`prisma.$queryRaw` / `drizzle.execute` accept raw SQL	Impossible — no SQL in application code. Xano API only.
Path Traversal	A01:2021	`fs.readFile(userInput)` in API routes or Server Actions	Impossible — no file system in V8 isolate.
Command Injection	A03:2021	`exec(userInput)` in Node.js routes	Impossible — no `child_process` in V8 isolate.
SSRF	A10:2021	`fetch(userInput)` in Server Components → reads internal services	Mitigated — Worker has no internal network. All fetches go to public APIs.
Prototype Pollution	A08:2021	`Object.assign({}, userInput)` in shared Node.js process	Mitigated — V8 isolate disposes after each request. No persistent prototype.
Deserialization	A08:2021	RSC payload deserialization, `JSON.parse` of untrusted data	Reduced — no RSC payload. JSON.parse used but no code execution path.
Server-Side XSS	A03:2021	RSC renders user input → hydration mismatch → client-side XSS	Eliminated — no server-side rendering of user content. Worker returns JSON.
Session Fixation	A07:2021	Database sessions not rotated on auth events	Mitigated — JWT in httpOnly cookie, signed per-request. No server session store.
Timing Attack	A02:2021	String comparison of secrets in Node.js	Mitigated — `crypto.subtle.timingSafeEqual` available in Workers runtime.
Memory Leak / DoS	A05:2021	Global state accumulation in long-running Node.js process	Impossible — V8 isolate disposed after each request. No accumulation.
Env Var Exposure	A05:2021	`process.env` accessible in Server Components → accidental render	Impossible — Workers use `env` parameter per-request. No `process.env`.

CVE CLASS	OWASP CATEGORY	NEXT.JS + PRISMA RISK	CRM SYNC (WORKERS + XANO)
LFI (Local File Inclusion)	A01:2021	`require(userInput)` or `import(userInput)` in Node.js	Impossible — no dynamic `require` / `import` from file system.

11.7 Why Distributed Decentralized Build Resolves These Risks

The core insight is that a **distributed, decentralized build** doesn't just mitigate risks — it **eliminates risk categories** by removing the capabilities that make them possible:


MONOLITHIC SSR (Next.js + Prisma)
Single Node.js process handles:
├─ Rendering (RSC → HTML → hydration)
├─ API logic (Server Actions, API routes)
├─ Database queries (Prisma/Drizzle → SQL)
├─ Auth (NextAuth middleware)
├─ File I/O (uploads, cache, sessions)
├─ Process execution (if needed)
└─ State (global vars, module scope)
Compromise of ANY layer → access to ALL layers
SSRF → reads DB credentials from process.env
Server Action bug → arbitrary SQL via Prisma
Path traversal → reads .env file from disk
Prototype pollution → affects all concurrent requests
DISTRIBUTED BUILD (Workers + Xano + Webflow)
Webflow Extension (UI only):
├─ Renders components from compiled bundle
├─ Cannot access Worker secrets, Xano data, or file system
└─ Sandboxed in Webflow Designer iframe
Cloudflare Worker (Logic only):
├─ Routes requests, validates auth, enforces consent

```
│  ├─ Cannot access file system, processes, or raw sockets          │
│  ├─ Cannot run SQL (talks to Xano API, not database)              │
│  └─ V8 isolate — no shared state between requests                 │
│                                                                    │
│  Xano (Data only):                                                 │
│  ├─ Exposes API endpoints (not raw SQL)                           │
│  ├─ Validates schema at platform level                            │
│  ├─ Manages connections, backups, scaling                         │
│  └─ Cannot be reached except through authenticated API calls      │
│                                                                    │
│  Compromise of ONE layer → access to ONLY that layer              │
│  Worker SSRF → can call public APIs only (no internal network)    │
│  Xano API key leak → can call endpoints only (no raw SQL)         │
│  Extension compromise → can render UI only (no secrets, no data)  │
└────────────────────────────────────────────────────────────────────┘
```

THE BLAST RADIUS DIFFERENCE

SCENARIO	NEXT.JS + PRISMA BLAST RADIUS	WORKERS + XANO BLAST RADIUS
Single env var leaked	`DATABASE_URL` → full DB access. `NEXTAUTH_SECRET` → forge any session.	`XANO_API_KEY` → API endpoint access only. `ADMIN_KEY` → admin endpoints only. Neither gives raw DB access.
RCE achieved	Full server access: read files, spawn processes, pivot to internal network	V8 isolate: no files, no processes, no internal network. RCE scope = make `fetch()` calls for 30 seconds.
Dependency compromised	Malicious code runs in shared Node.js process: read env vars, exfiltrate data, persist	No runtime deps to compromise. Build-only deps affect CI, not production.
Auth bypass	Attacker accesses Server Actions + DB directly	Attacker can call Worker endpoints — but Worker still enforces consent checks before data push. Auth bypass ≠ consent bypass.

11.8 When Next.js RSC is the Right Choice

This comparison is not a universal recommendation against Next.js. Next.js RSC is better when:

SCENARIO	WHY NEXT.JS WINS
Content-heavy marketing site	RSC's streaming HTML is faster for initial paint than API-fetched content
Rapid prototyping	Full-stack in one repo, one language, one deploy — faster to ship MVP
Team has React-only expertise	Smaller learning curve than Workers + Xano + Webflow
SEO-critical pages	Server-rendered HTML with meta tags — Workers would need a separate rendering layer
Real-time collaborative UI	Next.js + WebSockets + shared state is better supported
Single-tenant internal tool	Security blast radius matters less; DX matters more

CRM Sync's architecture is optimized for **multi-tenant SaaS with PII, consent, and compliance requirements** — where the blast radius of a single vulnerability can affect thousands of users across multiple systems. The distributed model pays a DX cost (three systems to coordinate) to buy a security property (physical isolation between layers) that no amount of Next.js middleware can replicate.

12. Success Criteria

METRIC	TARGET
Auth gate coverage	100% of write endpoints require authentication
Consent enforcement	100% of outbound pushes check consent state before sending
Sync logging	100% of outbound pushes logged to stream-specific sync_log
UCP visibility	Users can see which systems have their data and last sync time
Config audit	Every config change logged with before/after diff and actor
Non-destructive ops	Zero data-loss incidents from deploy or config changes
Stream addition time	New CDP integration < 1 week (inherits security/logging infra)
Rollback time	Config rollback < 30 seconds (KV write, no redeploy)
Runtime dependencies	Zero third-party npm packages in production Worker
Partner offboarding time	< 30 seconds (single config toggle, no redeploy)
Credential blast radius	1 tenant per compromised credential (tenant-isolated KV)